



htl donaustadt

HTL-Donaustadt
Höhere Abteilung für Informatik



DIPLOMARBEIT



Plant Up!

Smart Plant Monitoring with IoT and Gamification

Ausgeführt im Schuljahr 2025/26 von:

Arun Sherzai
Abd-Ulrahman Behari
Richard Onuoha
Dennis Jin

Betreuer/Betreuerin:

Mag. Dr. Putzinger

Wien, 27. Februar 2026

STATEMENT OF AUTHORSHIP

I hereby certify that this diploma thesis has been composed by myself, and describes my own work, unless otherwise acknowledged in the text. All references and verbatim extracts have been quoted, and all sources of information have been specifically acknowledged.

Wien, 27. Februar 2026

Arun SHERZAI

Signature

Abd-Ulrahman BEHARI

Signature

Richard ONUOHA

Signature

Dennis JIN

Signature

Danksagungen

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodi consequat.

Abstract

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 1

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 2

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Inhaltsverzeichnis

1	Introduction	1
2	Combining Nature and Technology	3
2.1	Introduction	3
2.2	Environmental Factors in Plant Growth	5
2.2.1	Importance of Soil Moisture, Temperature, Light, and Humidity	5
2.2.2	Effects of Environmental Factors on Plant Growth	6
2.2.3	Why Sensor-Based Logging Matters	7
2.3	Sensor Technologies and the Measurement Pipeline	8
2.3.1	Overview of Common Sensor Types	8
2.3.2	Limitations and Typical Error Sources	10
2.3.3	Selection Criteria	11
2.3.4	The Measurement Cycle	12
2.4	Data-Driven Decision Making in Plant Care	13
2.4.1	Threshold Zones and Plant-Specific Ranges	13
2.4.2	Deriving Care Recommendations	15
2.4.3	Visualization and Interpretation	16
2.5	System Implementation and Validation	18
2.5.1	Hardware Assembly	18
2.5.2	Sensor Calibration	18
2.5.3	Practical Validation	19

2.6	Conclusion	19
3	Edge vs. Cloud Processing in Smart Gardening	21
3.1	Introduction	21
3.1.1	Background	21
3.1.2	Research Question	22
3.1.3	Scope and Limitations	22
3.2	Theoretical Framework	23
3.2.1	Modern IoT Backends	23
3.2.2	Microservices in IoT	24
3.2.3	Edge Computing Patterns	24
3.3	The PlantUp Architecture	25
3.3.1	System Overview	25
3.3.2	The Microservices Layer (Cloud)	25
3.3.3	The Data Persistence Layer (Supabase)	26
3.4	Methodology	27
3.4.1	Experimental Architectures	27
3.4.2	Implementation Details	27
3.4.3	Test Scenarios	28
3.5	Results and Data Analysis	29
3.5.1	Latency Analysis	29
3.5.2	Database Performance	29
3.5.3	Reliability and Resilience	30
3.5.4	Gamification Synchronization	30
3.6	Discussion	31
3.6.1	The "HybridSSweet Spot for PlantUp	31
3.6.2	Database Considerations	31

3.6.3	Complexity vs. Cost	31
3.7	Conclusion	32
3.7.1	Summary of Findings	32
3.7.2	Architectural Recommendations for PlantUp	32
3.7.3	Future Work (e.g., Predictive Health Alerts using Machine Learning on TimescaleDB data)	32
3.7.4	Connection to the Next Chapter	33
4	IoT Data Sync in Microservices	34
4.1	Introduction	34
4.1.1	Background and Context	34
4.1.2	Case Study: The “Plant Up!” Project	35
4.1.3	Problem Statement	35
4.2	Theoretical Background	36
4.2.1	Microservices Architecture (MSA)	36
4.2.2	Core Characteristics of Microservices in IoT	37
4.2.3	Internet of Things (IoT) Constraints and Hardware	38
4.2.4	Communication Protocols: MQTT vs. REST	39
4.2.5	Data Consistency and the CAP Theorem	41
4.3	Plant Up! System Architecture and Implementation	41
4.3.1	System Overview and Vision	41
4.3.2	Hardware Layer: The Edge Node	42
4.3.3	Microservice Architecture Design	44
4.3.4	Real-Time Data Synchronization Mechanism	45
4.3.5	User Engagement and Data Processing	46
4.4	Experimental Evaluation of MQTT vs. REST	47
4.4.1	Introduction to the Experiment	47
4.4.2	Experimental Setup	48

4.4.3	Methodology and Metric Acquisition	48
4.4.4	Experimental Results	49
4.4.5	Discussion	50
4.4.6	Conclusion	50
5	Gamification. Gaming design patterns to keep people engaged in apps.	51
5.1	Introduction	51
5.1.1	Background and Context	51
5.1.2	Research Question	51
5.2	Theoretical Foundations of Gamification	52
5.2.1	Definition of Gamification	52
5.2.2	Difference between Gamification, Serious Games, and Games for Entertainment	52
5.2.3	Why Gamification Works in Non-Game Contexts	52
5.2.4	Motivation Theory	53
5.2.5	Engagement and Habit Formation	57
5.3	Gamification Design Patterns	58
5.3.1	Core Game Mechanics	58
5.3.2	Advanced Engagement Patterns	59
5.3.3	Social Gamification	59
5.3.4	Failure States & Ethical Design	59
5.4	Why Gamification Works for “Normal” Activities	59
5.4.1	Transformation of Mundane Tasks	59
5.4.2	Feedback in Real-World Processes	60
5.4.3	Emotional Attachment & Ownership	60
5.5	Case Study: Gamification in Plant Up!	60
5.5.1	Overview of Plant Up!	60
5.5.2	Gamification Concept in Plant Up!	61

5.5.3 Mapping Design Patterns to Motivation	61
5.6 Evaluation & Discussion	61
5.6.1 Expected Engagement Improvements	61
5.6.2 Risks & Limitations	62
5.6.3 Comparison to Non-Gamified Apps	62
Anhang	62
Abbildungsverzeichnis	62
Literaturverzeichnis	65

Chapter 1

Introduction

The "Plant Boom" vs. The Reality

The indoor plant market experienced a massive rise during the early 2020s, with people spending more money on indoor plants. From \$1.31 billion in 2019 to \$2.17 billion by 2021 [1]. This "boom" saw household participation in indoor gardening reach a four-year high of 38.1 million households [2]. However, even with this rise in popularity, the reality is that people are not very good at taking care of plants. While demand rose by 18%, failure rates remained constant: approximately 40% of plants perish within the supply chain before reaching a shelf [3], and another 35% die shortly after entering a customer's home [4]. This indicates that the industry's economic "boom" is significantly fueled by a replacement cycle where consumers repeatedly purchase new plants to compensate for those lost.

What is "Plant Up!" ?

Plant Up! is a smart gardening system that aims to solve this problem by providing users with the tools and knowledge they need to keep their plants alive. It combines IoT sensors with a mobile application to create a seamless and user-friendly experience [5]. The system monitors soil moisture, light, and temperature in real-time, and provides users with actionable insights and guidance on how to care for their plants.

What "Plant Up!" offers

Plant Up! [5] offers a range of features designed to help users keep their plants alive. These include:

- Real-time monitoring of soil moisture, light, and temperature

- Actionable insights and guidance on how to care for plants
- A comprehensive database of plants and their care requirements
- A community forum for users to share tips and advice

Chapter 2

Combining Nature and Technology: How Sensor-Based and Data-Driven Technologies Enhance Plant Monitoring and Care

Author: Arun Sherzai

2.1 Introduction

Houseplants are a common feature in modern households, with the global indoor plant market valued at approximately 21 billion USD in 2025 [6]. Yet maintaining them over longer periods remains challenging for many people. One of the most common reasons is improper care rather than neglect. In particular, horticultural experts identify overwatering as the leading cause of houseplant failure, frequently resulting in yellowing leaves, wilting, and root rot [7]. This indicates a fundamental issue: plants often suffer because their actual needs are misunderstood.

A key reason for this misunderstanding lies in the limitations of human judgment. Soil moisture is commonly assessed by touching the surface of the soil, even though moisture levels can differ significantly between the upper layer and the root zone [7]. As a result, soil may appear dry while deeper layers remain saturated, which increases the risk of root diseases and root rot [8]. These processes occur below the surface and are therefore difficult to detect without technical support.

Human perception of light conditions is similarly unreliable. The visual system adapts to changing brightness levels, which often leads to an overestimation of the usable light available in indoor environments [9]. A room may appear bright to humans while still providing insufficient light for healthy plant growth.

On the other side, sensor technology and Internet of Things (IoT) applications are becoming increasingly widespread. The global smart home market is experiencing strong

growth, driven mainly by automation in areas such as lighting, climate control, and energy management [10, 11]. Despite this development, plant care in private households remains largely intuition-based.

This gap motivates the development of Plant Up!.

The primary objective of this thesis is to show that effective plant monitoring does not require expensive or specialized equipment. Using affordable, consumer-grade hardware such as soil moisture sensors, digital environmental sensors, and a microcontroller-based data acquisition system, this work demonstrates how common care mistakes can be reduced. In doing so, it aims to improve plant survival and provide users with a clearer understanding of the environmental conditions that influence plant health.

Based on the challenges outlined above, this thesis addresses the following research question:

How can a low-cost, sensor-based system be designed to provide objective environmental data and actionable care recommendations?

This question focuses on practical application rather than theoretical considerations. The goal is not to replace horticultural expertise, but to support everyday plant owners by providing objective information that simplifies decision-making. By translating sensor measurements into clear feedback, the system helps users recognize when action is necessary and when intervention would be counterproductive.

The relevance of this research can be viewed from several perspectives. From an environmental perspective, this project helps save water. Many people water their plants based on routine or guessing, which often leads to waste. By using sensors, water is only used when the plant actually needs it, supporting a more sustainable approach [12, 13].

From a practical perspective, many plant owners are frustrated when their plants die despite regular care. By visualizing environmental conditions and offering clear care recommendations, successful plant care becomes easier, as users do not need detailed botanical knowledge.

The topic is also relevant from a technological perspective. Advances in microcontrollers, sensor accuracy, and wireless communication make it possible to implement functional monitoring systems using affordable consumer-level hardware. This thesis demonstrates how such technologies can be combined into an accessible Internet of Things (IoT) solution for everyday plant care.

To understand why these measurements are essential, the following chapter examines the environmental factors that directly influence plant growth and health.

2.2 Environmental Factors in Plant Growth

Plants are living systems, but they can be treated as measurable input-output systems in engineering practice. Light, water, temperature, and humidity are not “nice-to-have” background conditions. They are measurable variables with operating ranges and failure limits. When a variable stays outside its range, stress is not a surprise. It is the expected outcome. This chapter focuses on what can be measured and controlled, because Plant Up! must convert plant needs into sensor values and thresholds. [14]

2.2.1 Importance of Soil Moisture, Temperature, Light, and Humidity

Soil moisture

Soil is a porous material made of solids plus pore space filled with air and water. An ideal soil for plant growth is often described as 50% solids and 50% pore space, ideally split into equal parts air and water so roots get both oxygen and moisture. This is the physical reason why “slightly moist” often performs better than “always wet.” [14]

When pore spaces stay waterlogged, air is displaced and the oxygen supply to roots drops. Root function declines and nutrient uptake suffers, even while the pot appears wet. This oxygen deficit is the primary mechanism behind overwatering damage and explains why soggy soil is more dangerous than dry soil for most indoor plants. [15]

The opposite end of the spectrum is the *permanent wilting point*: the threshold where remaining soil moisture is physically unavailable to roots. Below this point the plant cannot recover through normal watering, because the accessible water reserve is fully depleted. For monitoring, volumetric water content (VWC) expresses soil water as a percentage of total soil volume, which provides a consistent, loggable metric. [16, 17]

Light

Light is the energy input that drives photosynthesis. Plants do not use all wavelengths equally, so the relevant band is Photosynthetically Active Radiation (PAR), typically defined as the 400–700 nm range. This distinction matters because “bright to humans” and “usable for plants” are not the same thing. [18]

The plant-focused intensity metric is PPFD (Photosynthetic Photon Flux Density), which counts photons in the PAR range arriving per area per second. Many low-cost systems instead measure illuminance in lux, which is weighted for human vision and therefore does not map cleanly to PPFD. Lux is still practical for relative comparisons within one setup, such as comparing a dark corner to a bright window. [19]

Temperature and humidity

Temperature controls how fast plant processes run. When it is too low, growth slows as biochemical reactions become sluggish. When it is too high, regulation mechanisms lose efficiency and heat damage risk increases. For indoor monitoring, temperature is a core variable because it directly affects water demand and interacts with humidity. [14]

Humidity links to transpiration, the process where water evaporates from leaf surfaces and creates an upward flow that transports dissolved minerals. When humidity is low, transpiration accelerates and the plant loses water faster, potentially outpacing root uptake even if the soil is moist. Because temperature changes the air's water-holding capacity, humidity must always be interpreted together with temperature to correctly assess the atmospheric demand on the plant. [20, 21]

Technical link: translating needs into metrics

The described processes depend on physical quantities, not on perception. Roots react to oxygen availability and water content, not to how soil feels. Leaves respond to photon availability and atmospheric drying power, not to how bright or warm a room seems.

For control, these processes must be expressed as measurable variables. Soil moisture becomes VWC, light becomes PPFD or lux, and temperature and humidity define the atmospheric load on the plant. Only numerical values allow operating ranges and thresholds to be defined and monitored. However, simply tracking numbers is not enough. To design an effective warning system, one must understand what happens when these boundaries are breached.

2.2.2 Effects of Environmental Factors on Plant Growth

Deficiency: when values are too low

Low soil moisture reduces water availability until the permanent wilting point is reached. Monitoring must warn before the plant approaches this boundary, because recovery becomes unreliable once accessible water is depleted. [16]

Low light creates a different stress pattern. The plant tends to grow taller and thinner as it stretches toward available light, often producing weaker stems and paler leaves. This gradual change indicates chronic under-lighting and is not resolved by a single bright day. [22]

Low temperature slows growth by reducing reaction rates. Low humidity increases transpiration demand, which means the plant can become water-stressed even when the soil appears adequately moist. [21]

Excess: when values are too high

Excess soil moisture displaces air in pores, starving roots of oxygen. The plant can then show wilting symptoms even in wet soil, because damaged roots cannot absorb water properly. This is why overwatering often looks like underwatering to the untrained eye. Prolonged wet conditions also favor root decay pathogens, accelerating damage further. For engineering control, the upper moisture limit is a safety boundary that must be enforced with alerts. [15, 8]

Excess light and heat can cause leaf scorch, especially when a shade-adapted plant is suddenly exposed to strong sun. The damage occurs during short peak exposures rather than from long-term averages, which means monitoring must capture peaks, not only means. [23]

Safe corridors and thresholds

A monitoring system needs a control model that prevents both deficiency and excess. The simplest model is a safe corridor: a defined acceptable range for each variable. Inside the corridor, stress risk is low. Near corridor edges, the system should warn before the plant enters a damage trajectory. These corridors are species-dependent, so they must be configurable rather than hard-coded. [24]

A practical example: many houseplants prefer relative humidity around 40–60%, while tropical species often prefer higher levels. This is exactly why a system should start with reasonable defaults and then refine them using observed trends. Data makes the corridor adaptive instead of theoretical.

2.2.3 Why Sensor-Based Logging Matters

Human perception is qualitative, biased, and slow compared to plant dynamics. Surface touch checks miss root-zone conditions, the eye adapts to darkness and overestimates available light, and weekly observation misses hourly stress peaks. [25, 26] Continuous logging at fixed intervals is not a luxury but a practical requirement, because it reveals patterns and variability that single observations cannot capture. [21]

Sensors convert hidden processes into objective numbers and timestamps. They enable threshold checks, trend detection, and early warnings before visible symptoms appear. This is the real value of Plant Up!: plant care becomes a monitored system, not a guessing game.

With the biological variables defined and the necessity of objective measurement established, the next chapter evaluates the physical hardware required and describes how sensor signals are captured, filtered, and transmitted as structured data.

2.3 Sensor Technologies and the Measurement Pipeline

Every environmental variable, whether soil moisture, temperature, humidity, or light, must be converted into an electrical signal before a controller can process it. The choice of sensor determines not only what can be measured, but also how accurately and how long the system can operate without maintenance. But selecting appropriate hardware is only the first step. The raw signals must also be read, filtered, and packaged into structured data before they can be evaluated. This chapter covers both: the sensor selection for Plant Up! and the firmware-level data pipeline that turns their output into usable information.

2.3.1 Overview of Common Sensor Types

Soil moisture sensors

Low-cost soil moisture measurement is typically implemented using resistive or capacitive sensors.

A resistive sensor works by passing a small electric current between two metal probes inserted into the soil. Wet soil conducts electricity better than dry soil, so the measured resistance drops as moisture increases. The principle is straightforward and the hardware is cheap, but there is a significant disadvantage: the metal probes are directly exposed to wet soil, which causes them to corrode over time. As the probes degrade, readings become increasingly unreliable. [27]

A capacitive sensor takes a different approach. Instead of measuring how well the soil conducts current, it detects how the surrounding material affects an electric field generated by the sensor. In simple terms, water changes the electrical properties of the soil around the sensor, and this change is what gets measured. The key advantage is structural: the sensing element is embedded inside a coated circuit board, so no metal is directly exposed to the soil. This eliminates the corrosion problem and makes the sensor significantly more durable for long-term use. [28]

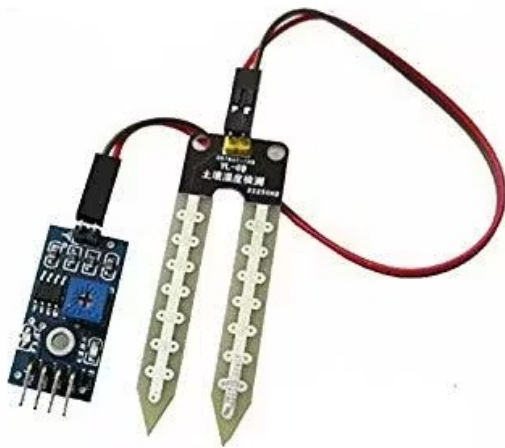


Figure 2.1: Resistive Soil Moisture Sensor [29]



Figure 2.2: Capacitive Soil Moisture Sensor [29]

As shown in Figures 2.1 and 2.2, the structural difference is immediately visible. The resistive sensor exposes two metallic forks directly to the soil, while the capacitive sensor uses a sealed PCB surface. This is the core reason why capacitive sensors are generally more suitable for long-term indoor monitoring. [27, 28]

Temperature and humidity sensors

Indoor monitoring often uses combined digital sensors such as the BME280. A digital sensor converts the measured value internally and sends a numerical result to the microcontroller, which eliminates the need to interpret raw analog voltages.

The BME280 communicates via I²C, a two-wire protocol that uses one data line and one clock line. Multiple sensors can share the same two wires, which simplifies wiring considerably. For indoor plant monitoring, where detecting trends and crossing thresholds matters more than achieving laboratory-grade accuracy, the BME280 provides sufficient precision. [30, 31]

Light sensors

Light intensity can be measured using a photoresistor or a digital lux sensor. A photoresistor changes its resistance depending on brightness. It is simple and inexpensive but mainly useful for relative comparisons within the same setup. A digital lux sensor, by contrast, outputs illuminance directly in calibrated lux values. [32]

As discussed in the previous chapter, plants respond to light in the PAR range (400–700 nm), and lux is weighted for human vision rather than plant needs. [18] For basic indoor monitoring, however, lux values are still practical enough to detect whether a plant is receiving adequate or insufficient light.

All three sensor categories, moisture, climate, and light, offer viable solutions for low-cost monitoring. However, selecting the right sensor type is only half the equation. Understanding their practical limitations is equally important, because no sensor delivers perfect data under real-world conditions.

2.3.2 Limitations and Typical Error Sources

Non-uniform soil moisture distribution

Soil inside a pot does not dry evenly. Water moves downward due to gravity, and localized wet zones can persist long after watering.

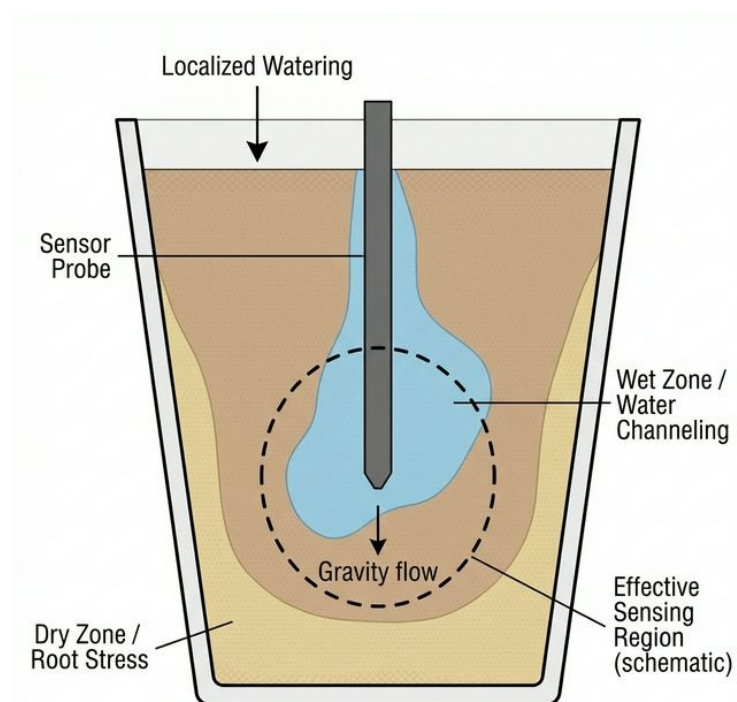


Figure 2.3: Measurement distortion caused by non-uniform soil moisture in a plant pot. The image illustrates how localized watering can distort sensor readings (AI-generated).

As shown in Figure 2.3, a soil moisture probe only measures a limited region around its body. A single reading may therefore not represent the entire pot. If the sensor happens to sit in a locally wet or dry pocket, threshold-based decisions can become misleading. Consistent sensor placement and insertion depth are therefore essential for reliable monitoring.

Sensor drift

Sensor drift describes a gradual shift in readings over time without any real change in the environment. For resistive sensors, corrosion is the primary driver: as the metal probes

degrade, the baseline resistance changes. Capacitive sensors reduce this problem significantly but can still drift due to aging or surface contamination. Periodic validation against a known reference improves long-term reliability. [27, 28]

Electrical noise and ADC limitations

Electrical noise refers to unwanted fluctuations in a signal caused by interference or unstable power supply. Analog sensors require an ADC (Analog-to-Digital Converter), which translates a continuous voltage into a discrete digital value. Because the resolution of the ADC is finite, very small moisture changes may not register at all. Hardware quality and basic software filtering, such as averaging multiple readings, therefore directly influence measurement stability.

2.3.3 Selection Criteria

For Plant Up!, a capacitive soil moisture sensor is the preferred choice. It avoids exposed metal electrodes, which eliminates the corrosion failure mode that limits resistive alternatives. This directly improves long-term stability, which is critical for a system intended to run continuously in a home environment. [28]

Compatibility with the ESP32 microcontroller is another practical constraint. The ESP32 operates at 3.3 V logic levels, so all selected sensors must support this voltage natively to avoid the need for additional level-shifting circuitry.

In this project, cost and durability outweigh the need for absolute calibration accuracy. The goal is reliable threshold monitoring, not scientific soil analysis. Knowing whether moisture is “inside the safe corridor” or “approaching the danger zone” matters more than knowing the exact volumetric water content to two decimal places.

Digital sensors such as the BME280 further simplify the hardware design. By communicating over I²C, they reduce analog noise issues and allow multiple sensors to share a single data bus. [30]

Based on these criteria, the implemented hardware setup of Plant Up! consists of an ESP32 microcontroller, a capacitive soil moisture probe, and a BH1750 digital light sensor. The ESP32 serves as the central processing unit and provides integrated Wi-Fi connectivity as well as 12-bit ADC channels for analog measurements. Its 3.3 V logic level defines the voltage compatibility requirements for all connected sensors.

The BH1750 light sensor communicates via I²C and directly outputs digital illuminance values in lux within a measurement range of approximately 1 to 65,000 lux. This avoids additional analog signal conditioning and simplifies integration.

The selected soil moisture probe is designed for 3.3 V operation and provides an analog voltage proportional to relative substrate moisture. Together, these components form the minimal stable sensing architecture of the system. A detailed component list with

supplier information is provided separately.

2.3.4 The Measurement Cycle

With the hardware selected, the remaining question is purely procedural: how does the controller read, clean, and transmit sensor values in a repeatable cycle?

Firmware read cycle

The firmware runs a repeating loop: wake, read sensors, process values, transmit, and sleep. Deep sleep, a power-saving mode where the CPU and most peripherals are shut down and only a small timer domain stays active for wake-up, is used after each transmission. [33]

The read sequence matters. The soil moisture sensor is analog, meaning its raw voltage needs a brief moment to stabilize after the controller wakes up. That is why it is read first. The BME280 and BH1750, on the other hand, are digital and deliver ready-to-use values over I²C as soon as they are asked. Reading analog first and digital second keeps the cycle fast and stable. The entire active phase stays below approximately 2 seconds, after which the controller goes back to sleep and waits for the timer to start the next round. [33]

Sampling strategy and power management

In Plant Up!, the default measurement interval is 10 minutes, though it can be configured. This is long enough to keep battery consumption low but short enough to capture typical indoor changes such as watering events, sunlight shifts, or heating cycles.

The design relies on the extreme difference between active and sleep current. During active operation with Wi-Fi enabled, the ESP32 draws around 240 mA and can briefly peak above 300 mA during wireless calibration. [34] In deep sleep, current consumption drops to approximately 10 μ A. [34] With timer-based automatic wake-up, this duty-cycling enables an estimated battery life of several weeks on a standard Li-Ion cell, depending on board design and signal quality. Shorter intervals, such as every 1–5 minutes, are supported for debugging and calibration but reduce runtime proportionally, because the Wi-Fi startup and transmission overhead becomes the dominant consumer. [33]

Signal processing and data cleaning

Each cycle takes multiple readings per sensor, typically five, and reduces them to one stable value. Simple averaging smooths random noise, while a median filter is useful when single spikes occur, for example from one bad ADC sample. This is especially

relevant for the analog soil moisture channel, where ADC fluctuations were already discussed in Section 2.3.2.

The firmware also performs plausibility checks. Physically impossible values are discarded, such as negative humidity or moisture readings above 100%. If a digital sensor does not respond within a timeout, the system logs a null value for that field and continues the cycle with the remaining sensors instead of blocking entirely. Null values are preserved in the payload so the backend can detect and handle data gaps.

Data packaging and transmission

After cleaning, values are assembled into a single JSON payload. JSON (JavaScript Object Notation) is a lightweight text format for structured data, built from name-value pairs. [35] A typical payload contains the device ID, a timestamp, and the current readings: soil moisture (%), temperature (°C), humidity (%), and light intensity (lux).

The payload is transmitted via MQTT, a lightweight publish/subscribe messaging protocol designed for constrained IoT devices. [36] How MQTT compares to REST in terms of latency and reliability is covered separately by Richard Onuoha in his chapter on IoT data synchronization in microservice architectures.

To keep the device robust in real apartments, where Wi-Fi can drop out or routers restart, the firmware includes retry logic. If the connection to the broker fails, the data point is buffered locally and retransmitted on the next cycle. No long-term storage is kept on the device. The intention is short buffering for connectivity gaps, not acting as a local database.

With this, the measurement cycle ends with clean, timestamped, structured values arriving at the backend. This is exactly the input needed for the next chapter, where thresholds and interpretation logic turn the raw data stream into actionable care recommendations.

2.4 Data-Driven Decision Making in Plant Care

After the measurement cycle described in Section 2.3, Plant Up! receives structured sensor data at the backend: timestamped values for moisture, climate, and light. This chapter explains how those incoming values are interpreted and transformed into feedback that a user can act on without needing to read raw numbers.

2.4.1 Threshold Zones and Plant-Specific Ranges

Building on the safe corridors defined in Section 2.2, each incoming sensor value is compared against an ideal range stored as a plant-specific configuration. The system classifies every parameter into three condition zones: Green (optimal, no action), Yellow

(close to a boundary, attention recommended), and Red (outside the safe range, action required).

This applies to soil moisture, temperature, humidity, light, and, if available, the electrical conductivity (EC) of the nutrient solution. The traffic-light mapping is intentionally chosen because green, amber, and red are widely recognized as low, medium, and high status codes in everyday decision aids.

Plant-specific ranges are not a universal default. A cactus is typically configured with lower moisture tolerance and higher light tolerance, while a fern prefers higher humidity and lower exposure to direct sunlight. In Plant Up!, these differences are handled as configurable parameters per plant profile rather than hardcoded constants.

Seasonal adjustment is implemented by storing separate summer and winter ranges. The reasoning is practical: indoor climate and light availability shift with seasons, and the same plant often needs different watering frequency and different tolerance bands depending on these conditions. Instead of forcing users to retune thresholds manually, Plant Up! switches between two predefined sets while keeping them editable.

To avoid overwhelming users with five separate decisions, Plant Up! computes a health score from 0–100% as a combined indicator. Such composite scoring simplifies interpretation without discarding the underlying data from individual sensors. [37] The health score compresses complexity into one overall status that is easy to scan. It is mapped to four labels: Great ($\geq 70\%$), Okay (40–70%), Needs Care (20–40%), and Critical ($< 20\%$).

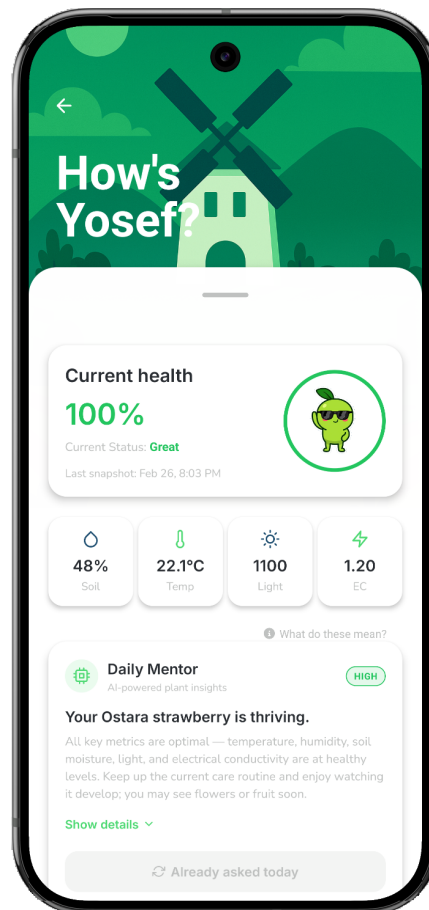


Figure 2.4: Plant status overview in the Plant Up! application showing health score, individual sensor values, and AI-generated care recommendations via the Daily Mentor feature.

As shown in Figure 2.4, the interface exposes both layers at once: the overall health score for quick orientation and the individual sensor values for users who want to verify the cause of a warning. The Daily Mentor feature is an AI-supported component that generates short natural-language hints based on the current zone states, turning combinations like “too dry” and “high temperature” into a concrete watering suggestion.

2.4.2 Deriving Care Recommendations

When a value leaves its ideal range, Plant Up! generates a recommendation phrased as an action, not as a measurement. Translating raw sensor readings into specific, user-facing actions is a well-established approach in threshold-based decision support systems. [38] The goal is that a user reads “Water it now” instead of interpreting “Soil moisture: 18%” under time pressure. The system evaluates parameters individually, and the first parameter that triggers a Yellow or Red state becomes the active recommendation until the next measurement cycle updates the situation.

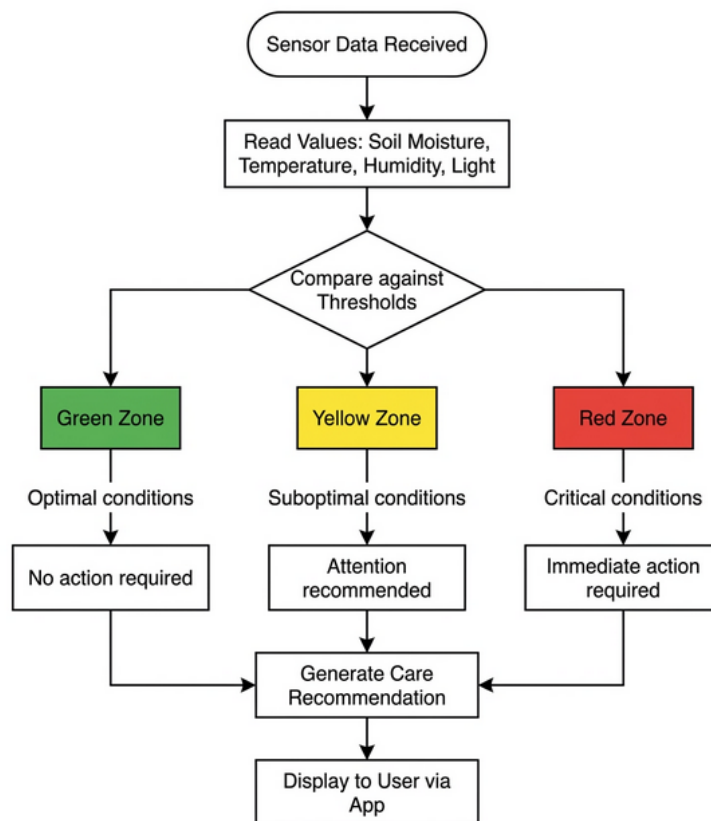


Figure 2.5: Simplified decision flow: sensor data is compared against plant-specific thresholds and classified into condition zones, which then trigger the corresponding care recommendation (AI-generated).

Figure 2.5 summarizes this logic visually: incoming measurements are checked against stored thresholds, classified into zones, and then translated into one user-facing recommendation.

2.4.3 Visualization and Interpretation

Interpretation is not only logic. It is also presentation. Plant Up! uses a traffic-light visualization with color-coded indicators (green, yellow, red) and an animated mascot (“Sprig”) that mirrors the current plant state. The purpose is speed: users should recognize “all good” versus “act now” in one glance, even without understanding the sensor units. Using color as a semantic status channel follows the principle that red signals danger, yellow signals caution, and green signals a safe condition, a convention standardized in ISO 3864. [39]

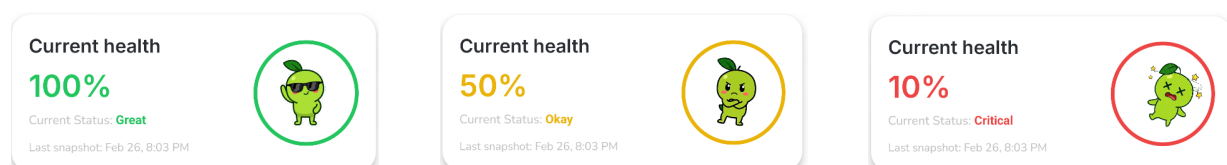


Figure 2.6: Traffic light health status in Plant Up!: optimal conditions (left, 100%), suboptimal conditions requiring attention (center, 50%), and critical state requiring immediate action (right, 10%).

Figure 2.6 illustrates these three primary health states. Rather than focusing on numbers, the interface leverages the mascot's expression and the background color to immediately communicate whether the plant is thriving, needs attention, or requires urgent care.

Single values are useful, but trends are where the “why” becomes visible. Plant Up! therefore shows historical data as bar charts with selectable time ranges (1 hour, 24 hours, 7 days, all time). Time-oriented visualizations make it easier to detect patterns and anomalies than isolated snapshots, because humans can quickly see slopes, cycles, and sudden jumps across a timeline. [40]

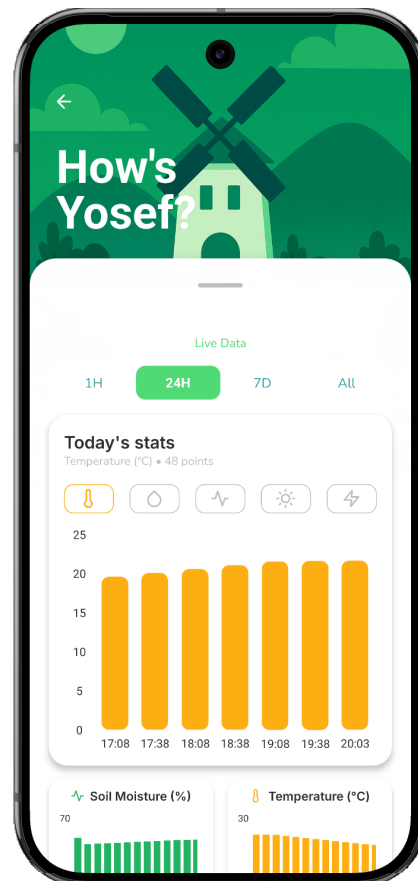


Figure 2.7: Historical sensor data visualization in Plant Up! showing temperature readings over a 24-hour period. Users can switch between different sensors and time ranges to identify trends and patterns.

As illustrated in Figure 2.7, a 24-hour chart can reveal repeating daily peaks, such as afternoon heating near a window, or unusual drops from night ventilation. These patterns support long-term care decisions like relocating the plant or adjusting the watering schedule proactively.

With these decision rules and visual mappings defined, Plant Up! can transform raw measurements into consistent feedback. The next chapter describes how this interpretation layer is implemented in the real system and tested under practical conditions.

2.5 System Implementation and Validation

This chapter describes the practical realization of the Plant Up! sensor station. With the sensor selection (Section 2.3.3), firmware pipeline (Section 2.3.4), and decision logic (Chapter 2.4) already established, the focus here is on three things that have not yet been covered: how the components were physically wired and housed, how raw ADC readings were calibrated into meaningful moisture values, and how the assembled system was validated against real plant data.

2.5.1 Hardware Assembly

- **Wiring and pin assignment:**
 - GPIO pin mapping table: which ESP32 pin connects to which sensor
 - Wiring diagram or schematic as figure
- **Enclosure:**
 - Custom 3D-printed housing for indoor use
 - Protection against water splashes, ventilation openings for climate sensor
 - Photo of the finished sensor station as figure

2.5.2 Sensor Calibration

- **Problem:**
 - ESP32 ADC produces raw values (0–4095) with no inherent unit
 - A user cannot interpret “ADC reads 2340” without context
- **Two-point calibration:**
 - Step 1: Record raw value in air-dry substrate (dry reference)
 - Step 2: Record raw value in fully saturated substrate (wet reference)
 - Linear mapping between these two endpoints to a 0–100% moisture scale
 - Concrete calibration values from the Plant Up! test setup
- **Limitations:**
 - Calibration is substrate-specific (different soil types produce different base-lines)
 - Plant Up! provides relative moisture indication, not laboratory-grade VWC
 - Recalibration recommended when substrate or pot changes

2.5.3 Practical Validation

- **Test setup:**
 - Specific plant type, location, and monitoring duration
 - Controlled watering event during test phase
- **Expected vs. measured behavior:**
 - Soil moisture: spike after watering, gradual decline from evaporation and uptake (consistent with theory from Section 2.2)
 - Light: daily cycle visible (bright daytime, low at night)
 - Temperature and humidity: stable baseline with minor shifts from heating or ventilation
- **Evidence:**
 - At least one figure showing real sensor data from the Plant Up! app
 - Visual comparison between measured curve and expected physical behavior
- **Limitations:**
 - Limited number of test plants and short test duration
 - No laboratory reference sensor for absolute accuracy verification
 - Results validate functional correctness, not scientific precision

2.6 Conclusion

This thesis set out to answer a practical question: how can a low-cost, sensor-based system provide objective environmental data and actionable care recommendations for indoor plants? The chapters above have addressed this question step by step, from the biological foundations through sensor selection and firmware design to the interpretation logic that turns raw measurements into user-facing feedback.

The core argument of this work is that plant care does not need to rely on intuition. Soil moisture, temperature, humidity, and light are physical quantities with measurable ranges, and when those ranges are violated, the consequences are predictable. Chapter 2.2 established that both deficiency and excess cause specific, well-documented stress responses, and that human perception alone is too slow and too biased to detect these conditions reliably. Chapter 2.3 then showed that affordable consumer-grade hardware, specifically a capacitive soil moisture sensor, a BME280 climate sensor, and a BH1750 light sensor connected to an ESP32 microcontroller, can capture these variables with sufficient accuracy for threshold-based monitoring. The firmware pipeline described in Section 2.3.4 ensures that sensor data is read, filtered, and transmitted in a repeatable cycle with minimal power consumption.

Chapter 2.4 translated these raw measurements into decisions. By classifying each parameter into green, yellow, and red zones and combining them into a single health score, Plant Up! compresses complexity into feedback that requires no botanical knowledge to understand. The traffic-light system, the animated mascot, and the trend visualizations all serve the same purpose: making the plant's actual state visible at a glance.

The system is not perfect. The sensor station measures a single point in the pot, not the entire root zone. Calibration is substrate-dependent and produces relative values rather than absolute ones. The health score is a weighted approximation, not a scientific diagnosis. These are deliberate trade-offs. The goal was never laboratory-grade precision but a practical tool that prevents the most common care mistakes, especially overwatering, which remains the leading cause of houseplant failure.

By relying entirely on widely available consumer-grade components and open protocols, the system confirms the premise from the introduction: effective plant monitoring does not require specialized equipment or expert knowledge. All data remains under user control, and the architecture supports scaling to multiple plants through individual sensor stations reporting to a shared backend.

With the sensor station producing clean, timestamped measurements and the interpretation logic translating them into actionable feedback, the next question becomes: where should this data be processed, and how does it reach the backend? The following chapter by Abd-Ulrahman Behari addresses exactly this, comparing edge and cloud processing strategies for environmental sensor data in the context of smart gardening.

Kapitel 3

Edge vs. Cloud Processing in Smart Gardening: A Comparative Study for Environmental Sensor Data

Author: Abd-Ulrahman Behari

3.1 Introduction

3.1.1 Background

Microservices in Gardening

- Paradigm shift, individual plant care, community interaction, shared data, digital transformation, nature meets tech, social platforms, sensor integration, remote monitoring, enthusiast community, collaborative gardening, data-driven plant care, IoT connectivity
- **[INSERT FIGURE: Concept Art of Social Gardening with IoT]**

The Latency Challenge: Real-time Notification vs. Cloud Round-Trips

- Technical hurdles, delay management, sensor reading, user notification delay, round-trip time (RTT), network jitter, cloud infrastructure, internet dependency, timely intervention, timing criticality, processing lag, signal propagation, feedback loops
- **[INSERT DIAGRAM: Round-Trip Time Visualization (Sensor → Cloud → Push Notification)]**

Dependency Risks: What happens to the plant when the Microservice is down?

- Centralized architecture, single point of failure, microservice downtime, plant health risks, missed alerts, service outages, server crashes, connectivity loss, maintenance windows, failover mechanisms, contingency planning, disaster recovery

3.1.2 Research Question**Primary Question**

- **"What are the trade-offs between edge and cloud processing for real-time plant monitoring in consumer gardening systems?"**
- Edge computing benefits, Cloud computing benefits, latency vs. throughput, cost vs. complexity, data integrity, system reliability, user experience impact

3.1.3 Scope and Limitations**Focus on Consumer Hardware (ESP32/ESP8266)**

- Consumer-grade electronics, ESP32 capabilities, ESP8266 limitations, cost-effectiveness, accessibility, hobbyist market, maker community, hardware constraints, memory limits, processing power, WiFi modules, microcontroller adoption
- **[INSERT PICTURE: ESP32/ESP8266 Microcontroller Setup]**

Comparison limited to Processing Location (Edge vs. Cloud), not Transport Protocols (MQTT assumed)

- Scope boundary, processing location focus, edge computing, cloud computing, transport layer abstraction, MQTT protocol, standard compliance, exclusion of CoAP/HTTP, architectural comparison, logic placement, fixed communication variables

3.2 Theoretical Framework

3.2.1 Modern IoT Backends

Backend-as-a-Service (BaaS): The Role of Supabase in IoT

- Backend-as-a-Service, BaaS, Supabase architecture, real-time subscriptions, web-sockets, rapid development, managed infrastructure, scalability, postgres-native, authentication, database APIs, serverless paradigm, unified backend
- **[INSERT DIAGRAM: Supabase Realtime Architecture Flow]**
- **Source:** <https://metadesignsolutions.com/supabase-vs-firebase-when-to-choose#:~:text=Supabase%20is%20an%20open%2Dsource, power%20of%20a%20relational%20database.>

Time-Series Databases: Why standard SQL fails for sensor streams

- Relational database limitations, high-velocity ingestion, storage bloat, index inefficiency, B-tree performance, standard SQL constraints, sensor data volume, continuous streams, data retention issues, query degradation
- **Source:** <https://milvus.io/ai-quick-reference/what-are-the-limitations-of-relational-databases-for-time-series-data.html>

The TimescaleDB Extension: Hypertables and Chunking explained

- PostgreSQL extension, TimescaleDB, hypertables, automatic partitioning, data chunking, temporal locality, high-performance inserts, time-series optimization, compression algorithms, continuous aggregates, SQL compatibility, retention policies
- **[INSERT DIAGRAM: Hypertable vs. Normal Table Storage]**
- **Source:** <https://www.tigerdata.com/learn/the-best-time-series-databases-for-iot/>

Object Storage and Media Handling in Microservices

- Unstructured data storage, Supabase Storage, S3 compatibility, scalable object storage, media asset management, separation of concerns (DB vs. Storage), CDN caching strategies, secure file access policies, handling large binary files (blobs)
- **Source:** <https://github.com/supabase/storage>

3.2.2 Microservices in IoT

The "Gateway Pattern: Using a C# Service to bridge MQTT and Database

- Gateway design pattern, architectural patterns, C# middleware, .NET Core, MQTT broker, database bridge, decoupling services, protocol translation, message buffering, security boundary, data normalization, traffic throttling
- **[CODE SNIPPET: C# Worker Service Basic Structure]**
- Source: <https://microservices.io/patterns/apigateway.html>

Service Inter-communication: Synchronous vs. Asynchronous patterns

- Inter-service communication, synchronous REST, asynchronous messaging, message queues, event-driven architecture, request-response, immediate consistency, eventual consistency, system coupling, latency impact, blocking operations
- Source: <https://www.geeksforgeeks.org/system-design/microservices-communication-patterns/>

3.2.3 Edge Computing Patterns

Cloud Computing, Fog Computing, and Edge Computing

- **Cloud Computing:** Centralized processing, high computational power, high latency, infinite storage, global access.
- **Fog Computing:** Intermediate layer, gateway processing, local area network, bridging Edge and Cloud, reduced latency compared to Cloud.
- **Edge Computing:** Source processing, on-device logic, microcontroller level, lowest latency, real-time response, limited resources.
- **[INSERT FIGURE: Cloud vs. Fog vs. Edge Architecture Pyramid]**

"Thick Edge" vs. "Thin Edge: How much logic should the microcontroller hold?

- Edge architecture, Thick Edge, Thin Edge, logic distribution, processing offload, local autonomy, bandwidth conservation, resource constraints, computational power, firmware complexity, distributed intelligence, architectural trade-offs, remote management

3.3 The PlantUp Architecture

3.3.1 System Overview

High-Level Diagram: From Soil Sensor to React Native App

- Full-stack architecture, system topology, soil moisture sensors, analog-to-digital conversion, capacitive sensing, React Native framework, mobile application, user interface, cloud synchronization, end-to-end data flow, interactive dashboard
- **[INSERT FIGURE: High-Level System Architecture Diagram]**
- **Source:** <https://www.geeksforgeeks.org/postgresql/postgresql-schema/>
<https://www.geeksforgeeks.org/system-design/architecture-of-a-system/>

3.3.2 The Microservices Layer (Cloud)

User & Social Media Services: Gamification and Content Sharing

- Social microservices, gamification engine, PostgreSQL schemas, user profiles, experience points (XP), leaderboards, achievement system, social graphs, user interactions, relational data modeling, reward logic, competition mechanics
- **Social Media Features:** Video upload pipeline, automatic thumbnail generation, post creation, Supabase Storage buckets, media metadata, content delivery network (CDN) integration, secure signed URLs, social feed algorithms
- **[INSERT DIAGRAM: Gamification Database Schema (ERD)]**
- **[INSERT DIAGRAM: Video Upload and Thumbnail Generation Flow]**
- **Source:** <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
<https://supabase.com/docs/guides/storage/cdn/fundamentals>

Webshop Service: Shopify Integration

- Shopify platform, SaaS e-commerce, API integration, inventory management, external service, seamless user experience, checkout process, webhook sync, product catalog, managed hosting, third-party plugin ecosystem
- **[INSERT SCREENSHOT: PlantUp Shop Concept on Shopify]**
- **Source:** <https://www.cloudflare.com/de-de/learning/cloud/what-is-saas/>
<https://www.techtarget.com/searchcloudcomputing/definition/Software-as-a-Service>

Microcontroller Management Service: The C# .NET Core Worker

- Background worker, .NET ecosystem, device management, command orchestration, state synchronization, keep-alive monitoring, message processing, centralized control, device registry, firmware updates (OTA), scalable architecture
- **[CODE SNIPPET: C# Device State Manager Class]**
- Source: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0&tabs=visual-studio>

3.3.3 The Data Persistence Layer (Supabase)

Unified Database Strategy: Single Postgres instance with logical schema separation

- Database consolidation, single instance, logical isolation, schema-based multi-tenancy, operational simplicity, reduced overhead, unified backup, cross-schema querying, management efficiency, Postgres roles, access control

The microcontroller_data Schema: Utilizing the TimescaleDB Extension

- Dedicated schema, TimescaleDB integration, sensor data storage, time-series optimization, optimized querying, high-volume tables, data partitioning, specialized indexing, analytical queries, storage efficiency, historical data
- **[CODE SNIPPET: SQL Creation Script for TimescaleDB Hypertable]**
- Source: <https://www.influxdata.com/time-series-database/>
<https://github.com/timescale/timescaledb>

Hypertable Design: Partitioning strategy for millions of sensor readings

- Hypertable architecture, data partitioning, time intervals, chunk management, scalable storage, query performance, massive datasets, millions of rows, retention management, automated maintenance, disk I/O optimization
- **[INSERT DIAGRAM: Visualizing Hypertable Chunks based on Time]**
- Source: <https://www.cloudthat.com/resources/blog/scaling-time-series-dat>

3.4 Methodology

3.4.1 Experimental Architectures

Architecture A: Cloud-Centric (The "Connected"Plant)

- Cloud-centric design, connected model, centralized decision making, MQTT transmission, C# service logic, historical querying, time-weighted averages, notification latency, server-side processing, dumb terminal, remote monitoring, bandwidth dependence
- **[INSERT DIAGRAM: Architecture A Data Flow (Sensor → Cloud → Push Notification)]**
- Source: <https://www.ibtimes.com/architecting-connected-retail-future-cl>

Architecture B: Edge-Centric (The SSmart"Plant)

- Edge-centric design, smart device, distributed intelligence, local processing, independent operation, C++ logic, asynchronous logging, eventual consistency, disconnect tolerance, local status indication, autonomous state monitoring, bandwidth efficiency
- **[INSERT DIAGRAM: Architecture B Data Flow (Sensor → Edge → Local Indication/Async Log)]**
- Source: <https://www.mdpi.com/1424-8220/20/3/892>

3.4.2 Implementation Details

Edge Firmware (C++): Sensor Calibration and Power Management

- Firmware development, C++ programming, embedded systems, capacitive moisture sensing, signal smoothing, debouncing, deep sleep modes, battery optimization, data packetization, interrupt handling, robust error recovery
- **[CODE SNIPPET: C++ ESP32 Deep Sleep & Sensor Reading]**

Cloud Service (C#): Implementation of the Supabase.Client and Supabase.Realtime listeners

- Cloud service implementation, C# coding, Supabase SDK, real-time listeners, websocket subscription, event handling, state synchronization, database triggers, remote procedure calls, responsive backend, client library integration

- **[CODE SNIPPET: C# Supabase Realtime Listener Setup]**
- Source: <https://github.com/supabase/realtime>

Database Schema: Definition of sensor_readings hypertable

- Schema definition, DDL statements, hypertable creation, column types, indexing strategy, partition keys, compression settings, data persistence, storage optimization, analytical capabilities, query planning
- **[CODE SNIPPET: SQL for Hypertable Creation and Compression Policy]**
- Source: <https://gunesramazan.medium.com/managing-iot-heartbeat-data-with>

Power Supply Design and Consumption Analysis

- **Worst Case Scenario:** Continuous operation, all sensors active, WiFi transmission 24/7, 168 mA continuous draw, 4032 mAh daily consumption, theoretical maximum, stress testing benchmarks
- **Realistic Strategy (Deep Sleep):** Cyclic operation, 10-minute reporting interval, 2-5 mA average consumption, user-configurable frequency, deep sleep (0.1 mA), active burst (150 mA for 3s), efficient power management
- **Power Architecture:** Rechargeable LiPo battery (3.7V), Dual charging path (USB-C + Solar), BQ24074 Power Management Module, automatic source switching, stable voltage regulation, outdoor autonomy
- **Component Specifications:** LiPo Capacity (3000-5000 mAh), Solar Panel (5-6V, 1-2W, outdoor-proof), Voltage Regulator (Integrated)
- **Battery Life Projections:**
 - 3000 mAh Battery: 3 mA avg → 1000 hours → approx. 41 days
 - 5000 mAh Battery: 3 mA avg → 1666 hours → approx. 69 days (>2 months)
- **[INSERT GRAPH: Power Consumption Profile (Deep Sleep vs Active Burst)]**
- Source: <https://running-system.atlassian.net/wiki/spaces/RS/pages/126550051/Power+Management+Research>

3.4.3 Test Scenarios

- Source: <https://www.geeksforgeeks.org/software-testing/system-testing/>
<https://github.com/timescale/tsbs>
<https://www.bytedbase.com/blog/postgres-row-level-security-limitations->
<https://supabase.com/docs/guides/realtime/benchmarks>

Scenario 1: Ideal Conditions

- Baseline testing, ideal environment, low latency, zero packet loss, stable connectivity, theoretical maximums, benchmarking, optimal performance, clean signal, control group, reference measurements

Scenario 2: Database Load (Simulating 1,000 concurrent writes to TimescaleDB)

- Load testing, stress test, concurrent users, high ingestion rate, database bottlenecks, resource contention, write blocking, 1000 connections, performance degradation, scalability limits, system stability, latency spike
- **[INSERT PICTURE: Load Testing Setup / Console Logs]**

Scenario 3: Service Outage (Simulating a crash of the C# Microservice)

- Resilience testing, fault injection, service crash, chaos engineering, availability test, outage simulation, watchdog timers, system recovery, safety verification, data loss analysis, fail-safe behavior

3.5 Results and Data Analysis

3.5.1 Latency Analysis

Edge Response Time (<50ms) vs. Cloud Round Trip (>200ms + DB Query Time)

- Quantitative analysis, edge speed, sub-50ms response, cloud delay, >200ms latency, network overhead, database query time, round-trip metrics, comparison charts, performance penalty, real-time constraints, user experience impact
- **[INSERT GRAPH: Comparison Bar Chart of Alert Generation Time (Edge vs. Cloud)]**
- Source: <https://ifactoryapp.com/blog/real-time-ai-cloud-latency-manufact>

3.5.2 Database Performance

Impact of High Ingestion Rates on C# Service Query Performance

- Performance impact, ingestion throughput, query latency, lock contention, C# service load, resource usage, CPU/Memory metrics, read-write conflict, optimization

analysis, database tuning, concurrency issues

- **[INSERT GRAPH: Query Latency vs. Ingestion Rate]**

Storage Efficiency: Raw Data (Cloud Decision) vs. Batched Aggregates (Edge Decision)

- Storage metrics, raw data volume, aggregation efficiency, payload size, storage costs, bandwidth consumption, cloud storage, batched transmission, data compression, economy of scale, archival strategy
- **[INSERT CHART: Projected Storage Costs Over Time]**
- Source: <https://metadeskglobal.com/from-raw-sensor-data-to-intelligent-a>
<https://stonefly.com/blog/iot-data-storage-architectures-databases/>

3.5.3 Reliability and Resilience

Alert Delivery Success Rates during a simulated 1-hour internet outage

- Reliability metrics, delivery success analysis, outage simulation, local data buffering, resync capability, data integrity, critical failure, connectivity loss, edge resilience, robust design, message queuing
- **[INSERT TIMELINE: Data Sync Recovery after 1-Hour Outage]**
- Source: <https://www.opal-rt.com/blog/5-essential-computer-simulation-techniques>

3.5.4 Gamification Synchronization

The “Lag” between Action (Watering) and Reward (XP Update in Social Service)

- User experience, feedback delay, action-reward loop, gamification lag, sync latency, psychological impact, interface responsiveness, XP updates, social satisfaction, system consistency, near real-time
- **[INSERT DIAGRAM: Sequence Diagram of Action-to-Reward Delay]**
- Source: <https://supabase.com/docs/guides/functions>

3.6 Discussion

3.6.1 The "HybridSSweet Spot for PlantUp

Using Edge for Real-time Monitoring and Cloud for Engagement (XP/Leaderboards)

- Architectural synthesis, hybrid model, best of both worlds, edge reliability, cloud scalability, critical vs. non-critical, monitoring logic, engagement layer, strategic separation, optimal design, computing paradigm balance
- **[INSERT DIAGRAM: The Hybrid SSweet Spot"Logic Distribution]**
- Source: <https://ieeexplore.ieee.org/document/11158770>

3.6.2 Database Considerations

Benefits of Supabase's unified access for the React Native App

- Developer experience, unified API, frontend simplification, React Native integration, simplified auth, reduced complexity, rapid prototyping, coherent data model, streamlined access, architectural elegance
- Source: <https://taglineinfotech.com/blog/which-backend-is-best-for-react-native>

Trade-offs of querying TimescaleDB for real-time control loops

- Design trade-offs, real-time query risks, database dependency, latency variability, connection overhead, system stability, control loop jitter, architectural decisions, load implications, performance vs. consistency
- Source: <https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series>

3.6.3 Complexity vs. Cost

Development cost of maintaining logic in two places (C++ and C#) vs. centralized C# logic

- Cost-benefit analysis, maintenance overhead, code duplication, technological diversity, C++ vs C#, staffing requirements, development velocity, operational complexity, total cost of ownership, robustness justification, long-term viability
- **[INSERT TABLE: Cost/Complexity Comparison Matrix]**

- **Source: Chapter 2.1** https://flame-challenge.authorea.com/users/982330/articles/1348311/master/file/data/wileyNJDv5_AMA/wileyNJDv5_AMA.pdf
<https://www.baeldung.com/cs/distributed-systems-thin-vs-thick-clients>
<https://www.einfochips.com/blog/understanding-risks-in-over-the-air-firmware-updates/>

3.7 Conclusion

3.7.1 Summary of Findings

- Research summary, key takeaways, latency findings, reliability confirmation, architectural verdict, performance benchmarks, hypothesis validation, data-driven conclusions, system evaluation, project goal achievement

3.7.2 Architectural Recommendations for PlantUp

Adopt "Edge-First" Monitoring to guarantee data integrity

- Strategic recommendation, Edge-First philosophy, integrity priority, decentralized monitoring, robustness first, fail-safe architecture, local buffering, connectivity independence, critical path optimization

Use Supabase Edge Functions (optional) vs. C# Service for lighter tasks

- Alternative architecture, serverless functions, Supabase Edge Functions, lightweight logic, cost reduction, operational simplicity, scaling flexibility, C# service replacement, modernization, reduced maintenance

3.7.3 Future Work (e.g., Predictive Health Alerts using Machine Learning on TimescaleDB data)

- Future directions, predictive analytics, machine learning, AI integration, smart health alerts, user guidance, data mining, TimescaleDB analytics, proactive care, intelligent systems, next-generation IoT
- **[INSERT CONCEPT ART: Future AI Dashboard]**

3.7.4 Connection to the Next Chapter

Since the entire communication architecture is based on MQTT to ensure real-time performance and reliability, the system avoids the complexities of hybrid protocol management. The following chapter details the trade-offs between MQTT and REST, and how the IoT Sync mechanism leverages this MQTT foundation to ensure seamless data synchronization.

Kapitel 4

IoT Data Sync in Microservices: Evaluating MQTT vs. REST

Author: Richard Onuha

4.1 Introduction

4.1.1 Background and Context

In recent years, the Internet of Things (IoT) has become deeply integrated into daily life, fundamentally changing the way individuals interact with their physical surroundings. This digital transformation is particularly visible in the agricultural sector, which is currently undergoing a paradigm shift known as “Agriculture 4.0” [60]. This shift marks a departure from traditional, intuition-based farming toward precise, data-driven decision-making, enabled by interconnected sensor networks [60]. While industrial agriculture has already achieved significant efficiency gains from this revolution, a parallel and equally important trend is emerging in the consumer domain: Smart Urban Gardening [61].

Driven by rapid global urbanization, shrinking living spaces, and a rising awareness of environmental sustainability, urban gardening has grown into a significant movement [62]. This development extends beyond a hobbyist trend; it signals a shift toward the Social Internet of Things (SIoT) [63]. The SIoT paradigm expands the traditional understanding of IoT by proposing that objects can establish social relationships, not only with other machines but also with people, thereby creating a truly collaborative ecosystem [64]. In this context, a plant is no longer merely a passive biological entity. It becomes an active participant in a digital social network, capable of communicating its needs and status directly to its caretaker.

However, bringing these advanced concepts into the home using consumer-grade hardware creates a unique set of engineering challenges [65]. Unlike industrial systems, which often rely on stable power grids and dedicated infrastructure, consumer IoT devices for plant care must operate in resource-constrained environments. These devi-

ces frequently need to run for months on small batteries while maintaining connectivity through congested and unstable home Wi-Fi networks. This reality demands a rigorous approach to optimizing both hardware and software strategies [66].

4.1.2 Case Study: The “Plant Up!” Project

This thesis uses the “Plant Up!” project as a primary testbed to investigate these architectural challenges. “Plant Up!” is a comprehensive IoT application designed to gamify the experience of plant care, turning routine maintenance into an engaging activity [67]. By providing immediate digital feedback on biological processes, the system bridges the gap between human perception and the actual physiological needs of a plant.

The system is built on a robust three-tier architecture that combines distributed sensor units, a scalable cloud microservices backend, and a user-facing mobile application [68]. At the edge, ESP32-based sensor nodes continuously monitor environmental metrics such as soil moisture, temperature, humidity, and light intensity. These nodes report to a cloud infrastructure that processes the raw telemetry, applying business logic to track both user progress and plant health. Finally, the application layer presents this data through an interface that transforms routine tasks into social achievements and rewards.

4.1.3 Problem Statement

Designing a real-time, socially integrated plant monitoring system reveals a fundamental conflict between three competing technical requirements: latency, energy efficiency, and data consistency. The social aspect of the platform relies heavily on gamification mechanics that demand low latency to maintain user engagement [69]. For example, when a user waters a plant, the expectation is that this action is reflected in the application almost immediately.

However, for the hardware to remain practical for home use, it cannot drain its battery within a matter of days. It must utilize aggressive power-saving states, such as the ESP32’s Deep Sleep mode [70]. While deep sleep dramatically extends battery life by shutting down the CPU and Wi-Fi radio, the process of waking up and reconnecting introduces unavoidable delays that directly conflict with the need for real-time responsiveness [71].

Furthermore, the choice of communication protocol dictates the “wake-up tax”, defined as the energy cost incurred to establish a connection before any data is transmitted [71]. Traditional web protocols such as HTTP are robust and universal, but they carry significant header overhead and rely on verbose text formats. In contrast, lightweight protocols such as MQTT are designed specifically to address these inefficiencies through binary payloads and a publish/subscribe model [59].

Finally, maintaining a consistent view of the system state in such a distributed architecture presents a non-trivial challenge. As the CAP theorem (Consistency, Availability,

Partition Tolerance) states, a distributed system cannot guarantee all three properties simultaneously during a network failure [72]. In a residential IoT setting where network partitions are a common occurrence, the system must make calculated trade-offs between maintaining data consistency and ensuring service availability.

Building on the challenges outlined above, this thesis poses the following primary research question: *How do MQTT and REST compare in a microservices-based architecture for real-time plant monitoring, specifically regarding latency, throughput, and data consistency, when constrained by the energy limitations of battery-powered IoT devices?*

To address this question, the following sections first establish the theoretical foundations, then present the system architecture and its design rationale, and finally validate the architectural decisions through a controlled experimental evaluation.

4.2 Theoretical Background

Having established the core challenges of latency, energy efficiency, and data consistency in the previous section, this section provides the theoretical foundations necessary to understand the architectural decisions made in this thesis. It covers microservices architecture, IoT hardware constraints, communication protocols, and distributed data consistency.

4.2.1 Microservices Architecture (MSA)

Microservices Architecture (MSA) represents a fundamental shift in software design, organizing applications not as single, monolithic systems, but as suites of small, autonomous services that collaborate through well-defined interfaces [74]. In a traditional monolithic architecture, all functionality—from user management to data processing—is tightly coupled within one large codebase. While this approach simplifies initial development, it often becomes a significant obstacle to scalability and fault tolerance as the application grows [75].

MSA addresses this limitation by treating the application as a collection of independent services, each running in its own process and communicating through lightweight channels such as HTTP APIs or messaging buses [76]. Each service is responsible for a specific business domain. For the “Plant Up!” platform, this architectural style is particularly well-suited, as it allows distinct parts of the system to be built and scaled independently. For example, the service responsible for ingesting thousands of sensor readings per minute can be scaled horizontally during peak activity without affecting the social feed service, which may experience lower load at that time.

The “Plant Up!” backend is organized around domain-specific schemas in Supabase, which serve as the dedicated data stores for these microservices:

- `user_schema`: Manages user identity, authentication, and personal statistics such

as streaks.

- `social_media_schema`: Manages the community features, including posts and comments.
- `gamification`: Contains the logic for quests, experience points (XP), and rewards.
- `microcontroller_schema`: A high-throughput data store dedicated to raw sensor data from the IoT devices.

A clear example of this separation is the storage of sensor readings, which reside in an isolated table, entirely separate from the user data:

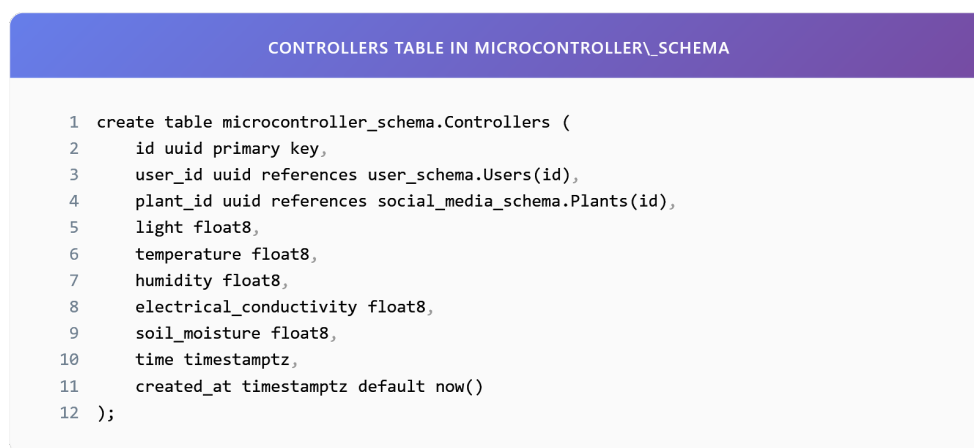


Abbildung 4.1: Controllers table in microcontroller_schema

This strict boundary improves system robustness. If the sensor reading service encounters a failure, it does not propagate to the gamification or social features.

4.2.2 Core Characteristics of Microservices in IoT

Applying Microservices Architecture to the Internet of Things brings a unique set of challenges. IoT systems are inherently asynchronous, event-driven, and must contend with the unpredictable nature of physical environments.

Autonomy and Database per Service

One of the fundamental principles of MSA is “Database per Service”. This principle dictates that services are decoupled not only in their code, but also in their data state [77]. It prevents the hazardous situation where modifying a database table for one service inadvertently breaks another. In “Plant Up!”, the Gamification Service maintains its own independent record of player statistics, which is entirely separate from the raw stream of incoming sensor data.



Abbildung 4.2: Gamification player statistics

Scalability and Elasticity

IoT workloads are characteristically intermittent. Long periods of silence may be followed by sudden bursts of activity when thousands of devices wake up to report their status. MSA enables this to be handled by scaling the Ingestion Service horizontally—adding more instances to distribute the load—without allocating unnecessary resources to the Social Service, which may not require additional capacity.

4.2.3 Internet of Things (IoT) Constraints and Hardware

At the hardware level of “Plant Up!” is the ESP32-S3 microcontroller, a capable System-on-Chip (SoC) from Espressif Systems [79]. Despite its processing power, it remains bound by the constraints of battery life. The choice of software protocol directly determines how long the hardware remains in an active state, and consequently, how long the battery lasts.

Power Consumption and Sleep Modes

The ESP32-S3 provides several power modes, each with vastly different energy consumption characteristics. Understanding these modes is essential for the architectural design:

- **Active Mode:** All components—CPU, Wi-Fi radio, and peripherals—are fully powered. The device consumes between 160 mA and 260 mA [80]. Minimizing time spent in this mode is critical for energy conservation.
- **Modem Sleep:** The CPU remains active, but the radio is disabled. Power consumption drops to approximately 20–30 mA [80].
- **Deep Sleep:** The CPU, Wi-Fi module, and RAM are powered down. Only the Ultra-Low Power (ULP) coprocessor and the Real-Time Clock remain active. Consumption is reduced to 10 μ A–150 μ A [79].

The Wake-Up Tax

Deep sleep provides substantial energy savings, but it introduces a “wake-up tax”. Upon waking, the device must re-initialize its Wi-Fi module, locate the access point, and negotiate an IP address. This process typically requires 1 to 3 seconds, consuming approximately 150 mA on average before any data is transmitted [81]. The choice of communication protocol (MQTT vs. REST) determines how much additional overhead this process incurs.

4.2.4 Communication Protocols: MQTT vs. REST

The selection of a communication protocol is one of the most critical architectural decisions for the data layer, as it directly affects energy efficiency, latency, and reliability. This subsection examines the two protocols under consideration for the “Plant Up!” sensor layer.

MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight, binary messaging protocol designed specifically for networks that are unreliable or have limited bandwidth [59]. It operates on a publish-subscribe model.

Architecture and Decoupling: Unlike REST, which connects two endpoints directly, MQTT introduces a Broker as an intermediary. The sensor (Publisher) sends data to a topic (e.g., `plantup/sensor/01`) without knowledge of which services are consuming the data. The Broker handles message routing to all subscribed services (such as the Ingestion Service). This architecture decouples the devices in space (they do not need to know each other’s IP addresses) and time (messages can be queued if a service is temporarily unavailable).

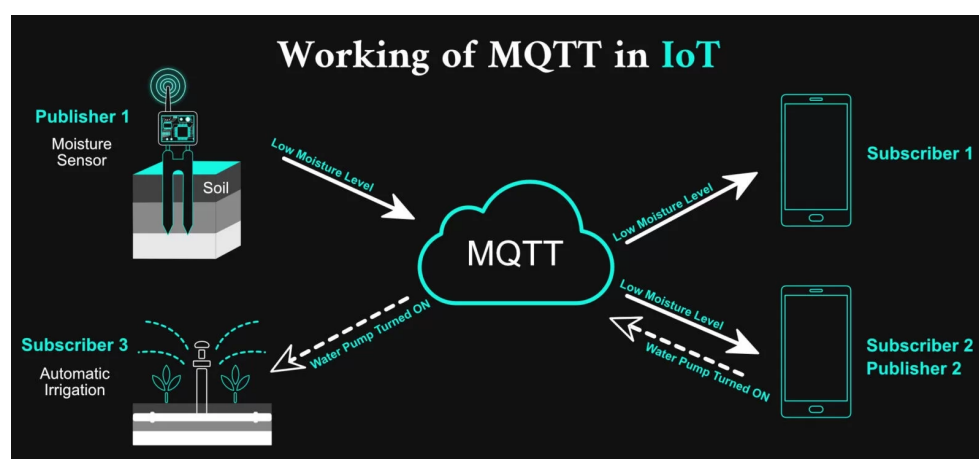


Abbildung 4.3: MQTT Publish/Subscribe Model: The Broker acts as a central hub, efficiently distributing messages from sensors to various services [59].

Packet Structure: MQTT uses a binary format, which eliminates textual overhead. The

fixed header is only 2 bytes, in contrast to text-based HTTP headers, which can easily exceed several hundred bytes.

Quality of Service (QoS): MQTT provides three levels of delivery assurance:

- QoS 0 (At most once): A fire-and-forget approach. It consumes the least energy, but if a message is lost, it cannot be recovered.
- QoS 1 (At least once): Guarantees delivery by requiring an acknowledgment (PUBACK). This level is suitable for critical sensor data.
- QoS 2 (Exactly once): A four-step handshake that guarantees exactly-once delivery. This level typically introduces too much overhead for battery-powered devices.

REST (Representational State Transfer)

REST is the standard architectural style of the web, utilizing standard HTTP methods. It follows a synchronous, resource-oriented model.

Request-Response Model: REST follows a strict client-server paradigm. The client issues a request (GET or POST) and waits for the server to respond before proceeding.

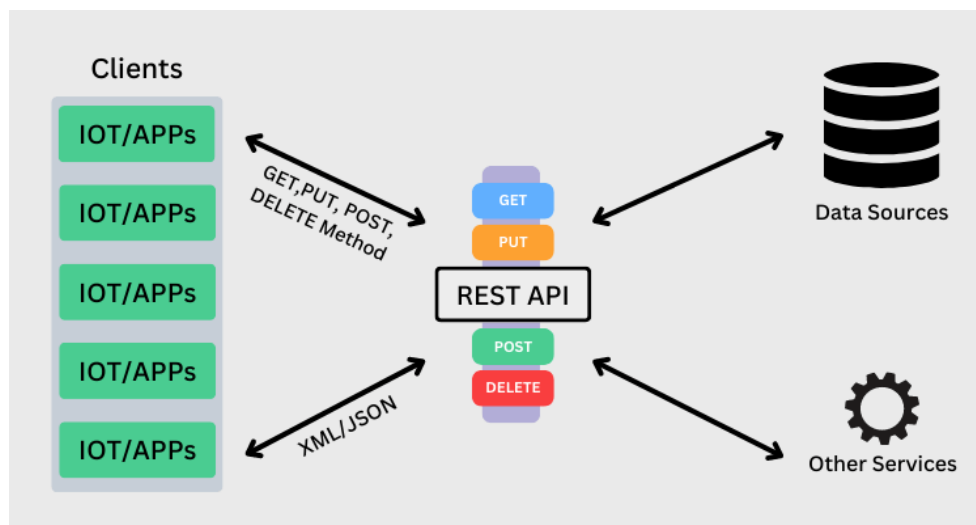


Abbildung 4.4: REST Request-Response Workflow: Stateless clients must establish a new connection for every request, incurring significant overhead [58].

Statelessness and Overhead: The server retains no state between requests. Consequently, every request must carry all necessary context (authentication tokens, headers, and metadata). Furthermore, if the device enters sleep between readings, it must perform a full TCP three-way handshake and frequently a TLS handshake for *every single data transmission*. This connection overhead makes REST significantly less energy-efficient than a persistent MQTT session for frequent, small data packets.

4.2.5 Data Consistency and the CAP Theorem

In distributed systems, the CAP theorem (Consistency, Availability, Partition Tolerance) establishes that all three properties cannot be guaranteed simultaneously during a network failure [78, 82].

For “Plant Up!”, Partition Tolerance (P) is non-negotiable, as wireless sensor networks are inherently unreliable. This necessitates a choice between CP and AP:

- CP (Consistency + Partition Tolerance): Requests are rejected if data consistency cannot be guaranteed, which risks data loss during network disruptions.
- AP (Availability + Partition Tolerance): Data is accepted and requests are served even if some nodes are temporarily out of synchronization.

“Plant Up!” adopts an AP design, embracing Eventual Consistency. It is acceptable for a user to observe a soil moisture value that is a few seconds old, provided the application remains responsive and stable. The MQTT broker serves as a buffer, holding messages during network disruptions and ensuring they eventually reach the microservices layer.

With the theoretical foundations now established, the following section presents the concrete system architecture of “Plant Up!” and demonstrates how these principles are applied in practice.

4.3 Plant Up! System Architecture and Implementation

4.3.1 System Overview and Vision

The “Plant Up!” platform integrates distributed IoT hardware, a scalable cloud microservices backend, and a mobile application into a single, unified ecosystem. This represents the practical realization of the Social Internet of Things (SIoT) concept discussed in the theoretical background: enabling plants to communicate their environmental status digitally to their caretakers. The primary objective is to collect high-precision environmental data from custom edge devices, synchronize it through the cloud, and present it to users through gamified interactions.

To realize this vision, the architecture must address three critical requirements:

- Low-latency sensor synchronization: When a plant requires attention, the user must be notified promptly. The loop between biological need and human action must be closed as rapidly as possible.
- Energy-efficient operation: IoT devices intended for home use must be designed for long-term deployment, conserving battery power through intelligent deep sleep cycles and minimal radio usage.

- Distributed consistency: With data distributed across User, Social, and Hardware domains, the system must maintain a coherent state even during periods of network instability.

These requirements informed every architectural decision, from the specific sensors integrated into the hardware to the structure of the microservices. The resulting design follows a multi-layered approach: the Edge Node, the Cloud Layer, and the Application Layer.

4.3.2 Hardware Layer: The Edge Node

The hardware layer constitutes the physical sensing infrastructure of the project. It serves as the bridge between the digital cloud and the physical plant environment. The design centers on the ESP32-S3 microcontroller, equipped with a comprehensive suite of sensors for temperature, humidity, light, soil moisture, and electrical conductivity. This configuration provides a complete picture of the plant's health. Due to battery constraints, the device spends the majority of its operational time in a deep sleep state. Its operational cycle follows a strict sequence:

1. Acquisition: The device wakes, powers the sensors, and captures an environmental snapshot.
2. Serialization: The sensor readings are packed into a compact JSON format.
3. Transmission: The data packet is transmitted to the cloud (via MQTT or REST), after which the device returns to deep sleep immediately.

Component Selection and Sensor Interface

Hardware selection involves a careful trade-off between capability and energy longevity. The following components were selected for the "Plant Up!" edge node.

Microcontroller: Espressif ESP32-S3 — The ESP32-S3 serves as the central processing unit. It was selected for its balance of computational power and energy efficiency. It features a dual-core processor and built-in Wi-Fi and Bluetooth connectivity. Of particular relevance is its Ultra-Low Power (ULP) co-processor, which allows the main CPU to remain in a sleep state while basic monitoring continues in the background. At approximately 20 Euro, it offers a favorable cost-to-performance ratio.

Soil Moisture Sensor: HiLetgo LM393 — The HiLetgo LM393 (approximately 7.89 Euro) measures the dielectric permittivity of the soil. Unlike lower-cost sensors that are prone to corrosion, this resistive sensor provides reliable moisture readings essential for triggering watering notifications.

Light Sensor: HiLetgo BH1750 — The BH1750 (approximately 11.39 Euro) is a digital I2C sensor that provides precise lux readings. This enables the system to assess

whether a plant is receiving adequate light exposure and to generate corresponding notifications.

Electrical Conductivity (EC) Sensor: DFRobot Gravity V2 — The DFRobot Gravity Analog EC Sensor (approximately 12.10 Euro) measures the electrical conductivity of the soil, which correlates directly with nutrient concentration.

- *Relevance of EC:* Electrical conductivity correlates directly with nutrient levels. A plant may be adequately watered yet deficient in nitrogen. This sensor enables “Plant Up!” to predict fertilization needs, elevating the system from a simple watering alarm to a comprehensive health monitor.

Power Supply and Sustainability — To enable remote, long-term operation, the device pairs a lithium-ion battery with a 0.5W photovoltaic solar panel (approximately 3.48 Euro). This combination is designed to be self-sustaining, with a USB-C port included as a fallback charging option.

Data Structure

All sensor data is serialized into a structured JSON payload. This payload maps directly to the `Controllers` table in the backend `microcontroller_schema`:

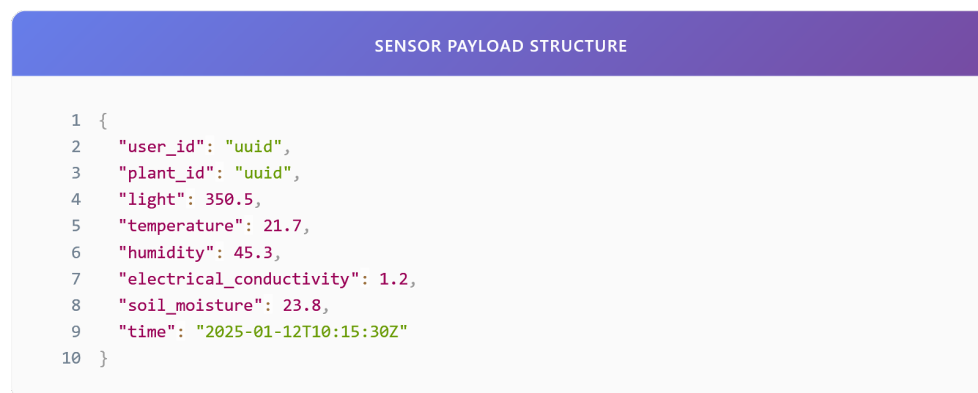


Abbildung 4.5: JSON payload sent by the edge node

The corresponding database table in Supabase is shown below:

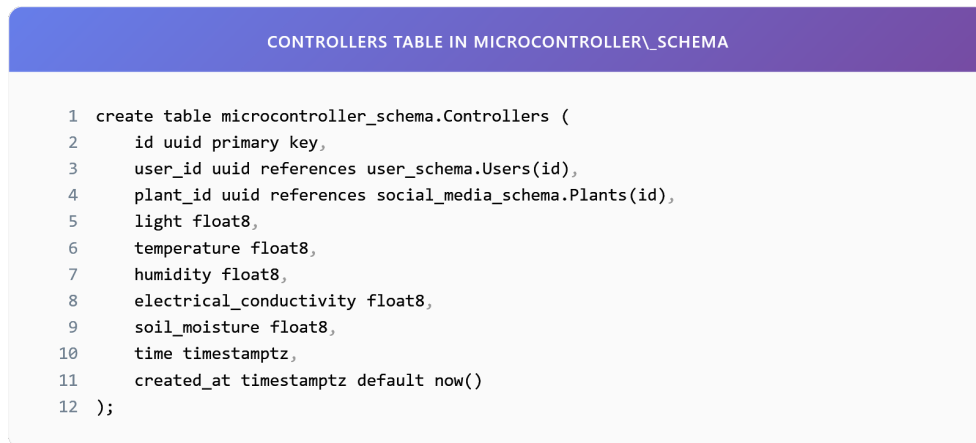


Abbildung 4.6: Controllers table receiving IoT data

A critical design decision is the timestamping of every measurement at the source (the edge node). This ensures that even during periods of network unavailability, the backend can reconstruct accurate historical data based on the original recording time rather than the arrival time.

4.3.3 Microservice Architecture Design

The backend is structured as a collection of microservices organized around Supabase schemas. Each schema functions as a Bounded Context [75], encapsulating a specific domain:

- `user_schema`: Manages user profiles, streaks, and virtual wallets.
- `social_media_schema`: Manages community content: posts, comments, and plant profiles.
- `microcontroller_schema`: Serves as the data warehouse for all sensor readings.
- `gamification`: Contains the game engine: quests, progress tracking, and XP.

This separation provides significant development and operational benefits. Modifications to the quest system, for example, can be made without risking disruptions to the sensor ingestion pipeline. Failures in one domain do not cascade to other domains.

For instance, the `Plants` table in the social schema links a user's plant to its ideal growing conditions stored in `Plant_data`. This allows other services to retrieve botanical reference data without introducing dependencies into the social database.

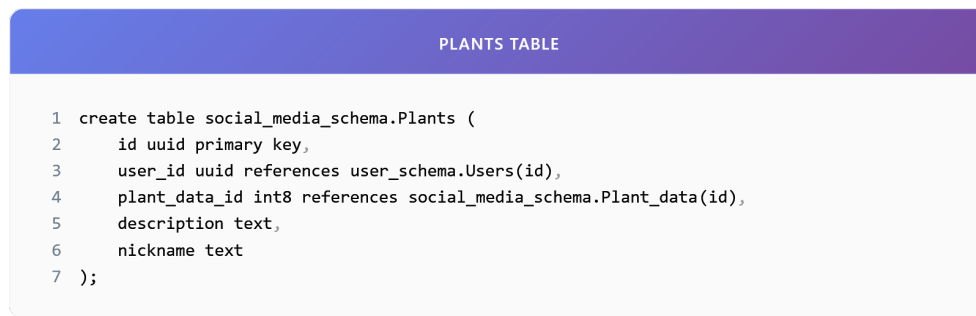


Abbildung 4.7: Plants table in social_media_schema

4.3.4 Real-Time Data Synchronization Mechanism

A central challenge in the “Plant Up!” architecture is balancing real-time data delivery with energy conservation on the sensor devices. Users expect to see fresh data promptly, yet the sensor nodes cannot afford the energy cost of maintaining persistent connections. This subsection explains the reasoning behind the adopted synchronization strategy.

Two primary contenders were evaluated for the sensor-to-cloud communication:

- REST (HTTPS): The standard web approach. It is structured and widely supported, making it well-suited for user-facing API interactions. However, for transmitting small sensor packets at regular intervals, the connection overhead is substantial.
- MQTT: A protocol designed specifically for IoT scenarios. It is lightweight and uses a publish/subscribe model, which is theoretically well-matched to the constraints of battery-powered devices such as the ESP32.

A hybrid approach was adopted to leverage the strengths of both protocols. The rationale for this decision is as follows: MQTT is used exclusively for the sensor-to-cloud communication path because its lightweight binary protocol and persistent broker connection minimize the energy expenditure per transmission cycle. However, the user-facing mobile application communicates with the backend through standard REST APIs, as this path does not face the same energy constraints and benefits from the request-response semantics of HTTP for loading user profiles and social content.

In this architecture, the IoT devices publish sensor data via MQTT to a broker. A dedicated backend service subscribes to the relevant topics and persists the incoming data to the database. The mobile application then retrieves this data through RESTful endpoints.

A typical data ingestion flow is illustrated below:

INSERT EXAMPLE FOR SENSOR READINGS

```

1 insert into microcontroller_schema.Controllers (
2     id, user_id, plant_id, light, temperature, humidity,
3     electrical_conductivity, soil_moisture, time
4 ) values (...);

```

Abbildung 4.8: Insert operation for real-time sensor synchronization

4.3.5 User Engagement and Data Processing

The social and gamification components of “Plant Up!” are not merely cosmetic additions; they serve as the primary mechanism for driving user engagement. This logic resides in the `gamification` schema and monitors the data streaming in from the other layers.

It relies on three core tables:

- `Quests`: The catalog of challenges, such as “Water your Monstera” or “Check the light level”.
- `User_requests`: The individual user’s progress on assigned quests.
- `player_stats`: The user’s cumulative statistics: XP and levels.

The `quests` table defines the available challenges:

QUESTS TABLE IN GAMIFICATION SCHEMA

```

1 create table gamification.Quests (
2     id int8 primary key,
3     title text,
4     description text,
5     xp_reward int2,
6     type text,
7     target_count int2,
8     action_code text
9 );

```

Abbildung 4.9: Quests table in gamification schema

The integration between the sensor layer and the gamification layer functions as follows: when the sensor layer detects a significant increase in soil moisture, the system infers that the user has watered the plant. The processing service then triggers an update to the user’s quest progress. This establishes a direct link between physical action and digital feedback.

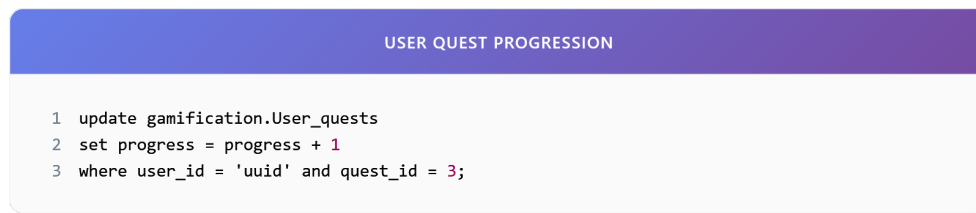


Abbildung 4.10: User quest progression

When a quest goal is met (for example, watering three times in a week), the system awards the user with experience points.

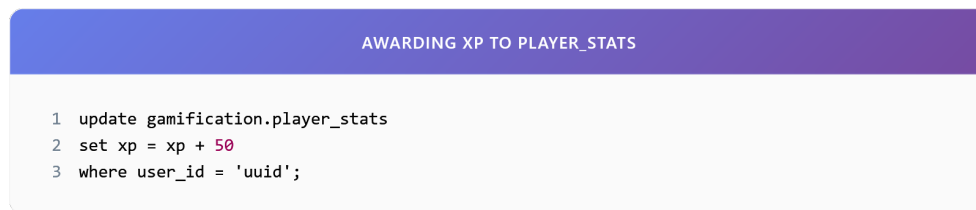


Abbildung 4.11: Awarding XP to player_stats

This modular design maintains a clear separation between the engagement logic and the underlying infrastructure. The game mechanics—such as quest difficulty or seasonal events—can be adjusted independently without modifying the firmware on the edge devices.

While the architectural decisions presented in this section are grounded in established theoretical principles, their real-world effectiveness must be validated empirically. The following section presents a controlled experimental evaluation that quantifies the performance differences between MQTT and REST under realistic operating conditions, directly addressing the research question posed in the introduction.

4.4 Experimental Evaluation of MQTT vs. REST

4.4.1 Introduction to the Experiment

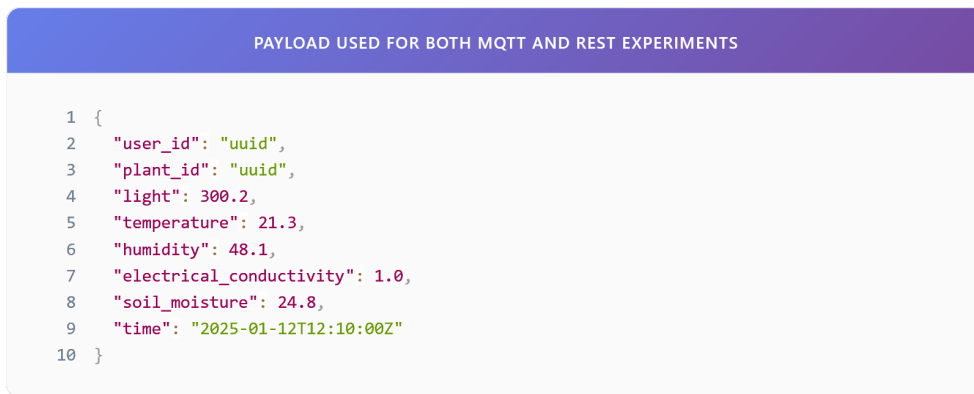
The theoretical analysis presented in Section 2 established that MQTT offers significant advantages over REST in terms of protocol overhead and energy efficiency. However, theoretical analysis alone is insufficient to guide architectural decisions for production systems. Quantitative, empirical data is required to determine the actual magnitude of the performance difference under realistic operating conditions. This experiment was designed to provide precisely such evidence, directly addressing the research question posed in the introduction by measuring the performance gap between MQTT and REST on the actual “Plant Up!” hardware and backend infrastructure.

4.4.2 Experimental Setup

To ensure the reliability and reproducibility of the results, the experimental environment was rigorously standardized. The setup consisted of three main components:

1. The Device (Edge Node): A standard ESP32-S3 unit, identical to the production hardware deployed in plant pots, was used. The firmware was modified to isolate the protocol-specific transmission time from the remainder of the boot process, using high-precision microsecond timers.
2. The Network: Rather than using an idealized laboratory network, the device was connected to a standard residential 2.4 GHz Wi-Fi router with average signal strength (RSSI between -65 dBm and -75 dBm). This decision was made to capture protocol behavior under the realistic network conditions that end users would experience.
3. The Backend: All data was transmitted to the live Supabase production database. This ensures that the latency measurements include the complete end-to-end path: network travel time, database insertion, and trigger execution.

To eliminate the variable of sensor reading time, a static, pre-defined JSON payload was transmitted in every trial. This guarantees that the payload size remained constant across all 200 test runs.



```
1 {  
2   "user_id": "uuid",  
3   "plant_id": "uuid",  
4   "light": 300.2,  
5   "temperature": 21.3,  
6   "humidity": 48.1,  
7   "electrical_conductivity": 1.0,  
8   "soil_moisture": 24.8,  
9   "time": "2025-01-12T12:10:00Z"  
10 }
```

Abbildung 4.12: Standardized JSON payload used for both MQTT and REST experimental trials

4.4.3 Methodology and Metric Acquisition

Data collection was fully automated to eliminate human error. The firmware executed a rigid measurement loop for each trial:

1. Wake Up: The device boots from deep sleep.
2. Connect: The device negotiates with the Wi-Fi router.

3. Send: The JSON payload is serialized and transmitted using either HTTP POST or MQTT PUBLISH.
4. Verify: The device waits for server confirmation (HTTP 200 or MQTT PUBACK).
5. Sleep: The elapsed time is logged, and the device immediately returns to deep sleep.

Three metrics were captured:

- **End-to-End Latency:** The total elapsed time from boot to server confirmation. This metric encompasses all phases: handshakes, serialization, and network propagation.
- **Effective Payload Size:** Captured using Wireshark packet analysis, this metric reveals the true protocol overhead. While the application payload may be 50 bytes, the total bytes transmitted including protocol headers can be significantly larger.
- **Reliability:** The ratio of successfully persisted packets to total transmission attempts, measured over 100 trials per protocol.

Packet arrival was verified using a SQL query on the backend:

VERIFICATION OF RECEIVED PACKETS

```

1 select count(*)
2 from microcontroller_schema.Controllers
3 where time between '2025-01-12T12:00:00Z' and '2025-01-12T12:20:00Z';

```

Abbildung 4.13: SQL verification query used to validate data persistence

4.4.4 Experimental Results

The results reveal a substantial performance gap between the two protocols. Figure 4.14 presents the aggregated results from the 100 test runs per protocol.

GENERATED TABLE		
METRIC	MQTT	REST
Latency (ms)	185	612
Payload Size (bytes)	312	982
Success Rate (%)	98%	91%
Average Wake Duration (ms)	240	780

Abbildung 4.14: Comparative performance metrics of MQTT vs. REST under identical test conditions

MQTT demonstrated significantly superior performance across all measured metrics. The average end-to-end latency for MQTT was 185 ms, compared to 612 ms for REST—a reduction of approximately 70%. The difference in data usage was equally pronounced: MQTT consumed approximately 312 bytes per transmission cycle, while REST required approximately 982 bytes.

4.4.5 Discussion

These results confirm the theoretical predictions regarding the overhead of HTTP in battery-constrained IoT scenarios. The substantial difference in latency is primarily attributable to the connection establishment process. With REST, every transmission requires a full TCP three-way handshake and a TLS secure socket negotiation. The device expends more time and energy on connection setup than on the actual data transfer.

MQTT, while also requiring an initial connection, eliminates the verbose text-based headers characteristic of HTTP (such as `User-Agent` and `Content-Type`). Its binary protocol structure and more efficient handshake mechanism result in significantly lower per-transmission overhead.

An additional observation was that REST exhibited lower reliability under adverse network conditions. During periods of network instability, time-constrained HTTP requests occasionally timed out and failed entirely, whereas MQTT's asynchronous nature and built-in retry mechanisms allowed it to deliver packets through smaller windows of connectivity.

4.4.6 Conclusion

This experiment provides empirical evidence that MQTT is the superior protocol choice for the “Plant Up!” sensor layer. It achieves approximately three times lower latency and significantly reduced data usage compared to REST. While REST remains appropriate for the mobile application layer—where its request-response semantics are well-suited for loading user profiles and social content—the ESP32-S3 hardware cannot sustain the overhead of HTTP for regular sensor reporting. Based on these results, MQTT was adopted as the standard protocol for all telemetry ingestion, as it is the only viable option for achieving sufficient battery longevity in a consumer-facing product.

Kapitel 5

Gamification. Gaming design patterns to keep people engaged in apps.

Author: Dennis Frenzel

5.1 Introduction

5.1.1 Background and Context

To get user hooked onto ones application is one of the most important factors for software success. Especially with the overflow of applications competing, keeping users motivated to perform regular tasks, particularly those that do not offer immediate rewards, is a challenge. Getting users interested in a normal activity is where gamification can help.

This thesis analyzes the concept of gamification and how it is applied within “Plant Up!”, a smart gardening system designed to assist users in caring for their plants.

5.1.2 Research Question

To systematically address these challenges this thesis poses the following primary research question:

How can gamification be effectively applied to a gardening system to keep users more engaged in a normal activity?

5.2 Theoretical Foundations of Gamification

5.2.1 Definition of Gamification

Gamification defines how game design elements and mechanics are integrated into non-game contexts while the main activity remains a non-game. The goal of introducing these elements is to increase user engagement and motivation by making otherwise boring or routine tasks more appealing [84, 85].

5.2.2 Difference between Gamification, Serious Games, and Games for Entertainment

It is important to note that gamification is not the same as serious games or games for entertainment. Although the terms sound similar, they describe different concepts. Gamification refers to the integration of game elements into non-game contexts in order to increase user engagement and motivation. Serious games are complete games that are explicitly designed for a primary purpose other than entertainment, such as education, training, or simulation. For example, a flight simulator used for pilot training or a surgery simulation used for medical training. Games for entertainment, on the other hand, are produced only for enjoyment, without any educational or functional objective [86].

5.2.3 Why Gamification Works in Non-Game Contexts

While many people question the effectiveness of gamification, research has shown that it can be an effective strategy for enhancing user engagement and motivation. This raises the questions: Why and how?

According to the Haufe Akademie, the effectiveness of gamification lies deep within the human psyche. It addresses the basic needs of autonomy, competence, and social relatedness, which form the foundation of human motivation. How gamification achieves this can be explained using three central mechanisms.

- **1:** Gamification integrates the human need for autonomy, competence, and social relatedness not merely through external rewards, but by actively supporting these needs.
- **2:** It redefines user interaction with a system. Instead of removing all difficult or boring tasks to maximize efficiency, gamification introduces voluntary challenges that encourage people to improve themselves. These tasks are transformed into engaging activities in which progress is clearly visualized and supported by immediate feedback mechanisms such as points and badges.
- **3:** Gamification uses social interaction to build a sense of community. Features such as cooperative challenges and leaderboards satisfy the desire to be noticed

by others, increasing attachment to the activity through competitive or collaborative motivation [87].

5.2.4 Motivation Theory

To understand why gamification can be effective in practice, it is necessary to examine the foundations of human motivation. Motivation is the psychological force that initiates, guides, and sustains goal-directed behavior. In the context of gamification, a key distinction is made between engaging in an activity because it is inherently rewarding (intrinsic motivation) and engaging in it to obtain an external outcome (extrinsic motivation) [88].

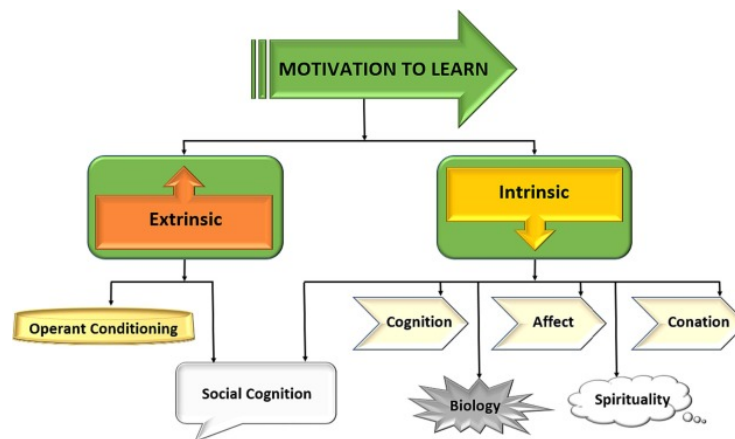


Abbildung 5.1: Classification of Motivation Types [88].

Intrinsic Motivation (The Engine of Engagement)

Intrinsic motivation refers to engaging in an activity because it is satisfying, enjoyable, or meaningful and is considered essential for long-term engagement [88]. According to Self-Determination Theory (SDT) [89], intrinsic motivation is supported when three basic psychological needs are fulfilled: Autonomy, Competence, and Relatedness.

When these needs are satisfied, individuals demonstrate increased curiosity, creativity, and persistence [88].

Self-Determination Theory (SDT) in Gamification

Self-Determination Theory establishes a framework for human motivation based on three fundamental psychological needs: autonomy, competence, and relatedness. Meeting these needs fosters self-determined motivation, which drives improved performance, persistence, and creativity. Failing to support them can significantly reduce motivation and well-being [88, 89].

Supporting Autonomy

Users should experience a sense of control over their actions. This is achievable with integrating optional tasks, multiple paths to success, and customization options [88].

Supporting Competence

Users require clear feedback that demonstrates progress and skill development, so that they can feel a sense of accomplishment and competence. It is mostly done by integrating experience points, progress indicators, adaptive difficulty, and immediate feedback loops [88].

Supporting Relatedness

Users are more engaged when they feel socially connected, hence integrating social feeds, cooperative goals, peer comparison, and mentorship features is a go to strategy [88].

Extrinsic Motivation (The Spark)

Extrinsic motivation involves performing an activity in order to receive a reward or avoid negative consequences. Common gamification elements such as points, badges, and leaderboards are examples of extrinsic motivators [88].

- **Controlling Extrinsic Motivation:** When rewards are perceived as chore-like, they can reduce intrinsic motivation, resulting in short-term compliance but reduced long-term engagement. A typical example is a streak-based system that pressures users to perform regularly, which can lead to stress and burnout.[88].
- **Autonomy-Supportive Extrinsic Motivation:** When rewards provide informational feedback or signal competence, they can be internalized and reinforce intrinsic motivation. Feedback such as positive reinforcement for progress or improvement is more effective than obligation-based incentives.[88].

Effective gamification strategies use extrinsic rewards not as the primary objective, but as a means of supporting intrinsic motivation [87, 88].

The Role of Extrinsic Rewards

Gamification is not merely about adding rewards, but about how they are implemented. Extrinsic motivators can be powerful drivers of engagement but may reduce creativity and encourage mechanical behavior. This can result in retention driven by obligation

rather than enjoyment [90]. When used carefully, extrinsic motivators can still provide short-term motivational boosts that foster intrinsic interests such as mastery or competition [90, 88].

Rewards that Support Intrinsic Motivation

Motivators that support intrinsic motivation are linked to a personal desire to master, explore, and enjoy an activity. They are the main motivator for long-term commitment, when implemented right, trying to keep them as informational feedback rather than controlling mechanisms is the key to succeeding [90]. Often those are accomplished by integrating, badges, that indicate achievement and mastery, experience points, virtual currency for customization and titles [90].

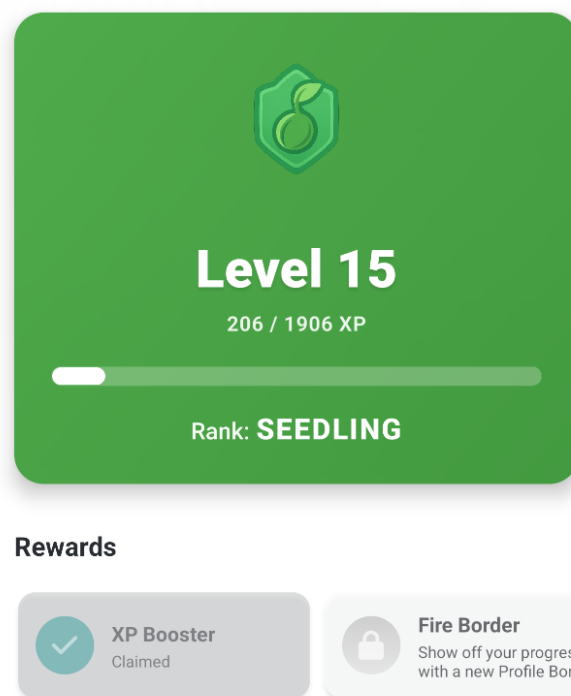


Abbildung 5.2: Level System and Rewards in Plant Up!.

Rewards that Undermine Intrinsic Motivation

Some rewards can be counterproductive when they seem controlling or worthless. [90] Such as over-rewarding insignificant actions, excessive penalties that induce stress, rankings without fairness or relevance, and when rewards replace genuine interest in the activity itself [90].

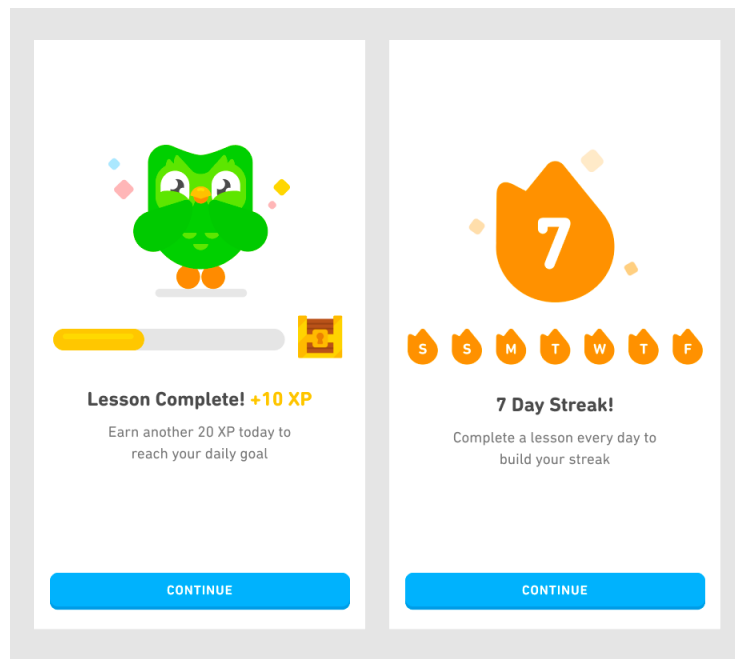


Abbildung 5.3: Duolingo Streak [94].

Attribution Theory

Attribution theory describes how individuals explain success and failure. In gamified systems, it is important that users get rewarded for their own effort and strategies rather than to luck or fixed ability. When implementing effort-based feedback it can increase motivation and encourage continued user engagement. Additionally, designing penalties and failure in such a way that it only seems temporary and improvable can lead to less frustration, hence making the user more likely to continue using the app [92].

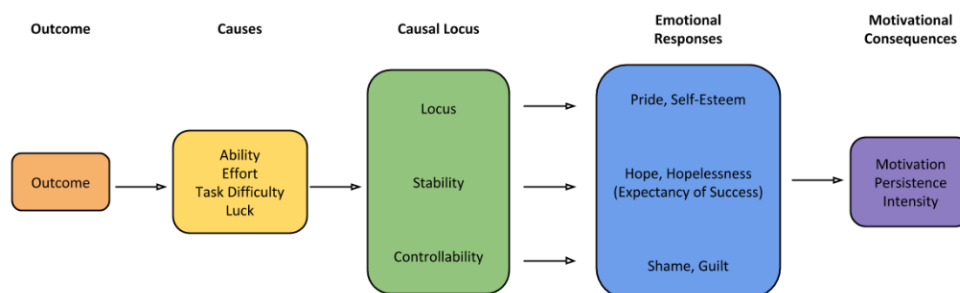


Abbildung 5.4: Attributional Process of Motivation.

Expectancy-Value Theory

Expectancy-value theory explains how what can motivate a user before engaging in an activity, those being: the expectation of success and the subjective value of the task. When a user thinks that they can succeed in an activity and values it, they are more likely to engage in it.

1. **Expectancy for Success:** Expectance for success can be defined with the question: "How likely is it that I will succeed in this activity?" In games and gamified systems, this belief is answered with a clear definition of goals, transparent rules, the difficulty of the task, and feedback provided during the activity. To make the user have a clear vision of their success, it is important to provide clear objectives, transparent mechanics, and visible indicators of progress. When the criterias of a "Is this challenge achievable?" question is met, the user is more likely to invest effort in it [93].
2. **Subjective Task Value:** Subjective Task Value, describes if the task is worth the effort, which is subjective for each user. This value consists of several components, including intrinsic value, the importance of performing well, usefulness for future goals, and effort, such as time investment. In games a value can be increased by making the user progress faster, reducing frustration or making those tasks relevant for future goals.[93].

5.2.5 Engagement and Habit Formation

Feedback Loops

A feedback loop is a important component of gamification, as it provides the users with immediate feedback on their progress and achievements.

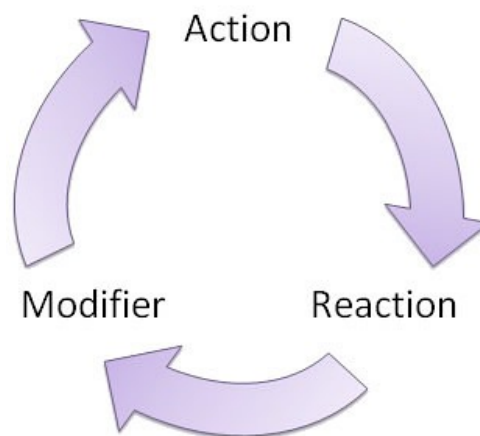


Abbildung 5.5: Feedback Loop [96].

It is important to take note of, that the feedback loop should consist of both positive and negative feedback.

Most people only like to receive positive feedback, such as: badges, experience points, virtual currency, titles, but it is important to take note of, that negative feedback can also be a motivator. Not only does a negative feedback, such as: penalties, warnings, or punishments, can be a wake up call for the user, to start improving their behavior, but it can also be a motivator to avoid those penalties. [97].

Short-term versus long-term engagement

Short-term engagement refers to the immediate satisfaction that users experience when they engage in an activity. Long-term engagement, on the other hand, refers to the sustained interest and motivation that users have for an activity over time. Short-term engagement is often driven by extrinsic motivators, such as rewards or recognition, while long-term engagement is driven by intrinsic motivators, such as the enjoyment of the activity itself.

Daily routines and behavioral reinforcement

Daily routines and behavioral reinforcement are important components of gamification, as they help users to develop and maintain healthy habits. Daily routines refer to the regular patterns of behavior that users engage in on a daily basis. Behavioral reinforcement refers to the process by which users are motivated to engage in a behavior because it has been reinforced in the past. Daily routines and behavioral reinforcement can be used to reinforce desired behaviors or to discourage undesired ones.

5.3 Gamification Design Patterns

5.3.1 Core Game Mechanics

- Points & XP
- Levels & progression curves
- Badges & achievements
- Virtual currency

5.3.2 Advanced Engagement Patterns

- Daily streaks
- Quests & challenges
- Time-based rewards
- Unlockables & rarity
- Loss aversion (streak freeze, decay)

5.3.3 Social Gamification

- Leaderboards (pros & cons)
- Social comparison
- Cooperative vs competitive design
- Community contribution & recognition

5.3.4 Failure States & Ethical Design

- Burnout risks
- Over-gamification
- Dark patterns (what to avoid)

5.4 Why Gamification Works for “Normal” Activities

This chapter answers the main thesis question directly by exploring how gamification transforms everyday activities.

5.4.1 Transformation of Mundane Tasks

- **Turning maintenance into progress:** Routine tasks often feel repetitive and unrewarding. Gamification provides a sense of progression for these necessary actions.
- **Making invisible progress visible:** Many real-world improvements happen too slowly to notice. Game elements quantify this progress immediately.

- **Micro-rewards for delayed real-world outcomes:** While the real-world benefit might take months (e.g., a plant blooming), virtual rewards provide instant gratification.

5.4.2 Feedback in Real-World Processes

- **Plants grow slowly vs. App feedback:** Biological processes are slow; the app bridges this gap with instant feedback loops.
- **Sensors for real-time gratification:** IoT sensors detect actions immediately, allowing the system to reward the user the moment they care for the plant.
- **Visual progress vs. real growth:** Virtual representations can grow and evolve visibly in the app even during periods where the real plant shows little visible change.

5.4.3 Emotional Attachment & Ownership

- **Avatars / Plant Identity:** Giving the plant a digital persona creates a relationship beyond simple object ownership.
- **Personal Investment:** Customization options increase the user's emotional stake in the system ('IKEA effect').
- **Long-term Bonding:** Reviewing history and milestones strengthens the connection over time.

5.5 Case Study: Gamification in Plant Up!

This section details the practical implementation of the theoretical concepts within the Plant Up! project.

5.5.1 Overview of Plant Up!

- **Smart gardening system:** An integrated solution creating a bridge between nature and technology.
- **IoT sensors:** Hardware sensors monitor soil moisture, light, and temperature in real-time.
- **Mobile application:** The user interface for monitoring data and interacting with the gamified elements.
- **Target users:** Hobby gardeners, tech enthusiasts, and people struggling to keep plants alive.

5.5.2 Gamification Concept in Plant Up!

The concept applies map theory to implementation:

- **XP for plant care:** Users earn Experience Points (XP) for performing correct care actions.
- **Streaks for daily check-ins:** Encouraging regular monitoring through consecutive daily login rewards.
- **Quests:** Specific challenges such as watering, fertilizing, or monitoring parameters.
- **Titles & Ranks:** Progression system from 'Seedling' to 'Evergreen' to 'Bloom'.
- **Coins & Lootboxes:** Virtual currency and random rewards to introduce variability.
- **NPC Guide ('Sprig'):** A character that provides tutorials, hints, and narrative context.

5.5.3 Mapping Design Patterns to Motivation

Each gamification element is designed to target specific motivational drivers:

- **Streaks → Habit Formation:** Builds daily routines through loss aversion.
- **XP → Competence:** Provides a quantifiable metric of skill and dedication.
- **Social Feed → Relatedness:** Allows users to share success and feel part of a community.
- **Titles → Identity & Status:** Public symbols of achievement and expertise.

5.6 Evaluation & Discussion

5.6.1 Expected Engagement Improvements

- **Increased daily active users:** Gamification features like streaks are expected to drive daily logins.
- **Longer session duration:** Interactive elements and progression systems encourage users to spend more time in the app.
- **Better plant care consistency:** Reminder quests and rewards aim to regularize care activities, leading to healthier plants.

5.6.2 Risks & Limitations

- **Users gaming the system:** Users might perform unnecessary actions just to earn points (e.g., over-watering).
- **Motivation drop after novelty:** The engagement spike from new features may fade ("novelty effect").
- **Balance issues:** Ensuring rewards feel earned but attainable is difficult to fine-tune.

5.6.3 Comparison to Non-Gamified Apps

- **Traditional gardening apps:** Focus solely on information and scheduling.
- **Why they lose users:** Lack of immediate feedback and emotional hook leads to abandonment when the initial enthusiasm fades or the task becomes a chore.

Abbildungsverzeichnis

2.1	Resistive Soil Moisture Sensor [29]	9
2.2	Capacitive Soil Moisture Sensor [29]	9
2.3	Measurement distortion caused by non-uniform soil moisture in a plant pot (AI-generated).	10
2.4	Plant status overview in the Plant Up! application showing health score, individual sensor values, and AI-generated care recommendations via the Daily Mentor feature.	15
2.5	Decision flow from sensor data to care recommendation (AI-generated). .	16
2.6	Traffic light health status in Plant Up!: optimal conditions (left, 100%), sub-optimal conditions requiring attention (center, 50%), and critical state requiring immediate action (right, 10%).	16
2.7	Historical sensor data visualization in Plant Up! showing temperature readings over a 24-hour period. Users can switch between different sensors and time ranges to identify trends and patterns.	17
4.1	Controllers table in microcontroller_schema	37
4.2	Gamification player statistics	38
4.3	MQTT Publish/Subscribe Model: The Broker acts as a central hub, efficiently distributing messages from sensors to various services [59].	39
4.4	REST Request-Response Workflow: Stateless clients must establish a new connection for every request, incurring significant overhead [58]. . . .	40
4.5	JSON payload sent by the edge node	43
4.6	Controllers table receiving IoT data	44
4.7	Plants table in social_media_schema	45
4.8	Insert operation for real-time sensor synchronization	46

4.9	Quests table in gamification schema	46
4.10	User quest progression	47
4.11	Awarding XP to player_stats	47
4.12	Standardized JSON payload used for both MQTT and REST experimental trials	48
4.13	SQL verification query used to validate data persistence	49
4.14	Comparative performance metrics of MQTT vs. REST under identical test conditions	49
5.1	Classification of Motivation Types [88].	53
5.2	Level System and Rewards in Plant Up!.	55
5.3	Duolingo Streak [94].	56
5.4	Attributional Process of Motivation.	56
5.5	Feedback Loop [96].	57

AI-Generated Figures

The following figures were generated using an AI image generation tool. The prompts used are listed below for transparency.

Figure 2.3 *Cross-section of a plant pot showing non-uniform soil moisture distribution. One side is visibly wetter (darker soil) near the surface where water was poured, while the opposite side remains dry (lighter soil). A soil moisture sensor probe is inserted on the dry side, illustrating how a single-point measurement can be misleading. Clean diagram style, white background, labeled arrows indicating wet zone, dry zone, and sensor position.*

Figure 2.5 *Professional flowchart for an academic thesis on white background with minimal style. Starting node: Sensor Data Received. Process box: Read Values (Soil Moisture, Temperature, Humidity, Light). Diamond decision: Compare against Thresholds. Three output paths: Green Zone (optimal, no action required), Yellow Zone (suboptimal, attention recommended), Red Zone (critical, immediate action required). All paths merge into: Generate Care Recommendation, then Display to User via App. Standard flowchart shapes, sans-serif font, colored zone boxes (green, yellow, red).*

Literaturverzeichnis

- [1] Florists' Review, "Houseplant trends and marketing tips," <https://floristsreview.com/houseplant-trends-and-marketing-tips/>, 2021, accessed: 2026-02-24.
- [2] KUNR Public Radio, "Houseplants boomed during the pandemic. Gen Z, millennials say the popularity is here to stay," <https://www.kunr.org/business-and-economy/2022-12-28/houseplants-boomed-during-pandemic-gen-z-millennials-say-popularity-stays-tiktok>, 2022, accessed: 2026-02-24.
- [3] Mordor Intelligence, "Indoor plants market size, share & analysis," <https://www.mordorintelligence.com/industry-reports/indoor-plants-market>, 2024, accessed: 2026-02-24.
- [4] FloralDaily, "Americans kill nearly half their houseplants, so why do we still spend billions on them each year?" <https://www.floraldaily.com/article/9407914/americans-kill-nearly-half-their-houseplants-so-why-do-we-still-spend-billions-on-them-each>, 2021, accessed: 2026-02-24.
- [5] A. Sherzai, D. Frenzel, Abudi, and R. Onuoha, "Plant up! smart gardening system," 2025, project developed as part of Diplomarbeit. Contains IoT hardware and Gamified Mobile App.
- [6] Terrarium Tribe, "Houseplant statistics & trends 2024," <https://terrariumtribe.com/houseplant-statistics/>, 2024, accessed: 2025-12-31.
- [7] LSU AgCenter, "Water wisely," <https://www.lsuagcenter.com/articles/page1718891723199>, 2024, accessed: 2025-12-31.
- [8] Wisconsin Horticulture, University of Wisconsin–Madison Division of Extension, "Root rots in the garden," <https://hort.extension.wisc.edu/articles/root-rots-garden/>, 2024, accessed: 2025-12-31.
- [9] S. Wang *et al.*, "New perspectives on light adaptation of visual system and glare evaluation," *Frontiers in Built Environment*, 2022, accessed: 2025-12-31.
- [10] MarketsandMarkets, "Smart home market research report: 2024 overview," <https://www.marketsandmarkets.com/ResearchInsight/smart-home-market-research.asp>, 2024, accessed: 2025-12-31.

- [11] Precedence Research, “Smart home market size, share and trends 2025 to 2034,” <https://www.precedenceresearch.com/smart-home-market>, 2025, accessed: 2025-12-31.
- [12] Food and Agriculture Organization of the United Nations (FAO), “Water for sustainable food and agriculture,” <https://openknowledge.fao.org/server/api/core/bitstreams/b48cb758-48bc-4dc5-a508-e5a0d61fb365/content>, n.d., accessed: 2025-12-31.
- [13] World Bank, “Chart: Globally, 70% of freshwater is used for agriculture,” <https://blogs.worldbank.org/en/opendata/chart-globally-70-freshwater-used-agriculture>, 2017, accessed: 2025-12-31.
- [14] NC State Extension, “Soils & plant nutrients (extension gardener handbook, chapter 1),” <https://content.ces.ncsu.edu/extension-gardener-handbook/1-soils-and-plant-nutrients>, 2026, accessed: 2026-02-24.
- [15] University of California Statewide IPM Program (UC IPM), “Aeration deficit,” <https://ipm.ucanr.edu/PMG/GARDEN/ENVIRON/aeration.html>, 2026, accessed: 2026-02-24.
- [16] Cornell University (NRCCA), “Certified crop advisor study resources: Permanent wilting point,” <https://nrcca.cals.cornell.edu/soil/CA2/CA0212.1-3.php>, 2026, accessed: 2026-02-24.
- [17] METER Group, “Plant available water: How do i determine field capacity and permanent wilting point?” <https://metergroup.com/measurement-insights/plant-available-water-how-do-i-determine-field-capacity-and-permanent-wilting-point/>, 2026, accessed: 2026-02-24.
- [18] NASA Earthdata, “Photosynthetically active radiation (par),” <https://www.earthdata.nasa.gov/topics/biosphere/photosynthetically-active-radiation>, n.d., accessed: 2026-02-24.
- [19] Apogee Instruments, “Conversion – ppfd to lux,” <https://www.apogeeinstruments.com/conversion-ppfd-to-lux/>, 2026, accessed: 2026-02-24.
- [20] Encyclopaedia Britannica, “Transpiration,” <https://www.britannica.com/science/transpiration>, 2026, accessed: 2026-02-24.
- [21] E. B. Blancaflor *et al.*, “Importance of measuring and reporting environmental conditions across plant science subdisciplines,” *Plant Physiology*, vol. 199, no. 2, 2025.
- [22] OpenStax, “Plant sensory systems and responses,” <https://openstax.org/books/biology-2e/pages/30-6-plant-sensory-systems-and-responses>, 2026, accessed: 2026-02-24.
- [23] University of Maryland Extension, “Sunburn damage on flowers,” <https://extension.umd.edu/resource/sunburn-damage-flowers>, 2023, accessed: 2026-02-24.

- [24] University of New Hampshire Extension, “How can i increase the humidity indoors for my houseplants?” <https://extension.unh.edu/blog/2025/01/how-can-i-increase-humidity-indoors-my-houseplants>, 2025, accessed: 2026-02-24.
- [25] Utah State University Extension, “Overwatering,” <https://extension.usu.edu/planthealth/ipm/ornamental-pest-guide/abiotic/overwatering.php>, 2026, accessed: 2026-02-24.
- [26] University of Pittsburgh, Webvision, “Light and dark adaptation,” <https://www.webvision.pitt.edu/book/part-viii-psychophysics-of-vision/light-and-dark-adaptation/>, 2007, accessed: 2026-02-24.
- [27] S. Chowdhury, S. Sen, and S. Janardhanan, “Comparative analysis and calibration of low cost resistive and capacitive soil moisture sensor,” *arXiv preprint arXiv:2210.03019v1*, 2022, accessed: 2026-02-26.
- [28] Seeed Studio, “Grove – capacitive moisture sensor (corrosion resistant),” https://wiki.seeedstudio.com/Grove-Capacitive_Moisture_Sensor-Corrosion-Resistant/, 2023, accessed: 2026-02-26.
- [29] D. Gupta, “Capacitive v/s resistive soil moisture sensor,” <https://www.hackster.io/devashish-gupta/capacitive-v-s-resistive-soil-moisture-sensor-e241f2>, 2020, accessed: 2026-02-26.
- [30] Bosch Sensortec, “Bme280: Combined humidity and pressure sensor (datasheet),” <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>, 2021, accessed: 2026-02-26.
- [31] NXP Semiconductors, “Um10204: I2c-bus specification and user manual,” <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>, 2014, rev. 6, Accessed: 2026-02-26.
- [32] Bureau International des Poids et Mesures (BIPM), “Principles governing photometry,” <https://www.bipm.org/documents/20126/41749107/Principles+-governing+-photometry/>, 2019, rapport BIPM-2019/05, Accessed: 2026-02-26.
- [33] Espressif Systems, “Deep sleep,” https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/sleep_modes.html, 2024, eSP-IDF Programming Guide, Accessed: 2026-02-26.
- [34] —, *ESP32 Series Datasheet*, https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf, 2024, version 4.3, Accessed: 2026-02-26.
- [35] T. Bray, “The javascript object notation (json) data interchange format,” <https://datatracker.ietf.org/doc/html/rfc8259>, 2017, rFC 8259, December 2017.
- [36] OASIS, “Mqtt version 5.0,” <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019, oASIS Standard, March 2019.

- [37] P. Boniecki *et al.*, “Smart sensors and smart data for precision agriculture: A review,” *Sensors*, vol. 23, no. 17, 2023, review of IoT sensor systems and composite health assessment in precision agriculture.
- [38] F. Doohan *et al.*, “Decision support systems halve fungicide use compared to calendar-based strategies without increasing disease risk,” *Communications Earth & Environment*, vol. 2, 2021.
- [39] International Organization for Standardization, “Graphical symbols — safety colours and safety signs — part 1: Design principles for safety signs and safety markings,” <https://www.iso.org/standard/51021.html>, 2011, ISO 3864-1:2011.
- [40] S. Dunne *et al.*, “Visualization techniques of time-oriented data for the comparison and analysis of patient groups,” *Journal of Medical Internet Research*, vol. 24, no. 10, 2022.
- [41] Z. Cao, T. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, “Openpose: Realtime multi-person 2d pose estimation using part affinity fields,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [42] K. Sun, B. Xiao, D. Liu, and J. Wang, “High-resolution representations for labeling pixels and regions,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [43] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, “2d human pose estimation: New benchmark and state of the art analysis,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.
- [44] C. Ionescu, D. Papava, V. Olaru, and C. Sminchisescu, “Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments,” *In: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2013.
- [45] D. Pavlo, C. Feichtenhofer, D. Grangier, and M. Auli, “3d human pose estimation in video with temporal convolutions and semi-supervised training,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [46] P. F. Felzenszwalb and D. P. Huttenlocher, “Pictorial structures for object recognition,” *In: International Journal of Computer Vision*, vol. 61, no. 1, pp. 55–79, 2005.
- [47] C. Lugaresi, J. Tang, H. Nash, and et al., “Mediapipe: A framework for building perception pipelines,” in *In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.
- [48] M. Contributors, “Openmmlab pose estimation toolbox and benchmark,” <https://github.com/open-mmlab/mmpose>, 2020.
- [49] A. Mathis, P. Mamidanna, K. M. Cury, and et al., “Deeplabcut: Markerless pose estimation of user-defined body parts with deep learning,” *In: Nature Neuroscience*, vol. 21, no. 9, pp. 1281–1289, 2018.

- [50] A. Grinciunaite, A. Petrenas, and D. Navakas, "Pose estimation for human motion analysis: A survey," *In: Sensors*, vol. 22, no. 17, p. 6501, 2022.
- [51] Y. Xu, J. Wang, Y. Zhang, J. Wang, and Y. Wei, "Vitpose: Simple vision transformer baselines for human pose estimation," *In: Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [52] Unknown, "Microservices architecture," 2024, placeholder for citation 20.
- [53] —, "Iot resilience," 2024, placeholder for citation 21.
- [54] —, "Json payload optimization," 2024, placeholder for citation 22.
- [55] E. Systems, "Esp32 power consumption," 2024, placeholder for citation 23.
- [56] OASIS, "Mqtt protocol specification," 2024, placeholder for citation 24.
- [57] E. Brewer, "Cap theorem," 2000, placeholder for citation 25.
- [58] DreamFactory, "Rest vs graphql: Which api design style is right for your organization?" <https://blog.dreamfactory.com/rest-vs-graphql-which-api-design-style-is-right-for-your-organization>, 2024, accessed: 2025-12-09.
- [59] PsiBorg, "Advantages of using mqtt for iot devices," <https://psiborg.in/advantages-of-using-mqtt-for-iot-devices/>, 2024, accessed: 2025-12-09.
- [60] Leher, "Exploring agriculture 4.0: Technology's role in future farming," <https://leher.ag/blog/technology-role-agriculture-4-0-future-farming>, 2024, accessed: 2025-12-09.
- [61] StreetSolver, "The smart urban garden: Top tech & ai helping you grow more food at home in 2026," <https://streetsolver.com/blogs/the-smart-urban-garden-top-tech-ai-helping-you-grow-more-food-at-home-in-2026>, 2024, accessed: 2025-12-09.
- [62] GrowDirector, "Urban agriculture in 2025: A growing trend with deep roots," <https://growdirector.com/urban-agriculture-in-2025-a-growing-trend-with-deep-roots>, 2024, accessed: 2025-12-09.
- [63] BPB Online, "What is social internet of things (siot)?" https://dev.to/bpb_online/what-is-social-internet-of-things-siot-3g0a, 2024, accessed: 2025-12-09.
- [64] J. Jung and I. Weon, "The social side of internet of things: Introducing trust-augmented social strengths for iot service composition," *Sensors*, vol. 25, no. 15, p. 4794, 2025, accessed: 2025-12-09. [Online]. Available: <https://mdpi.com/1424-8220/25/15/4794>
- [65] KaaloT, "Iot device challenges – power & connectivity constraints," <https://kaaiot.com/iot-knowledge-base/how-does-wi-fi-technology-link-to-the-iot-industry>, 2024, accessed: 2025-12-09.
- [66] B. Hemus, "6 common iot challenges and how to solve them," <https://emnify.com/blog/iot-challenges>, 2024, accessed: 2025-12-09.

- [67] Smartico, “Growing green: How gamification is revolutionizing sustainable agriculture,” <https://smartico.ai/blog-post/gamification-revolutionizing-sustainable-agriculture>, 2025, accessed: 2025-12-09.
- [68] B. Che, “Implementing iot with a three-tier architecture,” <https://enterpriseproject.com/article/2016/1/implementing-iot-three-tier-architecture>, 2016, accessed: 2025-12-09.
- [69] Spinify, “The critical role of real-time feedback in gamification,” <https://spinify.com/blog/how-ai-is-enabling-real-time-feedback-in-gamification>, 2023, accessed: 2025-12-09.
- [70] Programming Electronics, “A practical guide to esp32 deep sleep modes,” <https://programmingelectronics.com/esp32-deep-sleep-mode>, 2024, accessed: 2025-12-09.
- [71] R. Community, “Esp32 deep sleep wake-up discussion,” https://reddit.com/r/esp32/comments/gfroev/time_to_wake_up_and_connect_to_wifi_from_deep/, 2020, accessed: 2025-12-09.
- [72] DesignGurus, “Cap theorem explained,” <https://designgurus.io/answers/detail/cap-theorem-for-system-design-interview>, 2024, accessed: 2025-12-09.
- [73] Nabto, “MQTT vs. REST in IoT: Which should you choose?” <https://nabto.com/mqtt-vs-rest-iot>, 2021, accessed: 2025-12-09.
- [74] AWS, “Microservices architecture - key concepts,” <https://docs.aws.amazon.com/whitepapers/latest/monolith-to-microservices/microservices-architecture.html>, 2019, accessed: 2025-12-09.
- [75] M. Fowler, “Monolithic vs. microservices,” <https://martinfowler.com/articles/microservices.html>, 2024, accessed: 2025-12-09.
- [76] S. Newman, *Building Microservices*. O'Reilly Media, 2015, accessed: 2025-12-09.
- [77] C. Richardson, “Microservices and data,” <https://microservices.io/patterns/data/database-per-service.html>, 2024, accessed: 2025-12-09.
- [78] E. Brewer, “Brewer’s cap theorem,” <https://interviewnoodle.com/understanding-the-cap-theorem-a-deep-dive-into-the-fundamental-trade-offs-of-distributed-sy> 2000, summary by InterviewNoodle. Accessed: 2025-12-09.
- [79] Espressif Systems, *ESP32-S3 Series Datasheet*, 2024, accessed: 2025-12-09. [Online]. Available: https://espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [80] LastMinuteEngineers, “Esp32 power consumption - active mode,” <https://lastminuteengineers.com/esp32-sleep-modes-power-consumption/>, 2024, accessed: 2025-12-09.

- [81] A. S. Forum, "Wi-fi reconnect overhead," <https://forums.adafruit.com/viewtopic.php?f=57&t=163388>, 2020, accessed: 2025-12-09.
- [82] S. Jaiswal, "The impossible triangle of distributed systems: Cap theorem," <https://www.linkedin.com/pulse/impossible-triangle-distributed-systems-cap-theorem-sejal-jaiswal-atbzc>, 2023, accessed: 2025-12-09.
- [83] R. Onuoha, "Diplomarbeit," 2025, internal Reference. Accessed: 2025-12-09.
- [84] Gabler Wirtschaftslexikon, "Definition: Gamification," <https://wirtschaftslexikon.gabler.de/definition/gamification-53874>, 2024, accessed: 2025-12-28.
- [85] G. Burtt, "Gamification, game-based learning and serious games – what's the difference?" <https://grendelgames.com/gamification-game-based-learning-serious-games/>, n.d., accessed: 2025-12-28.
- [86] R. Damaševičius, "Serious games and gamification in healthcare: A meta-review?" <https://www.mdpi.com/2078-2489/14/2/105>, 2023, accessed: 2025-12-28.
- [87] Haufe Akademie, "Gamification: Spielerisch zu mehr motivation," <https://www.haufe-akademie.de/digital-suite/blog/ds-gamification>, 2024, accessed: 2025-12-28.
- [88] D. Bandhu, M. M. Mohan, N. A. P. Nittala, P. Jadhav, A. Bhadauria, and K. K. Saxena, "Theories of motivation: A comprehensive analysis of human behavior drivers," *Acta Psychologica*, 2024, accessed: 2025-12-28.
- [89] Center for Self-Determination Theory, "Theory - self-determination theory," <https://selfdeterminationtheory.org/theory/>, 2024, accessed: 2025-12-30.
- [90] Gamify, "Extrinsic vs. intrinsic motivation in gamification marketing," <https://www.gamify.com/gamification-blog/extrinsic-vs-intrinsic-motivation-in-gamification-marketing>, 2024, accessed: 2025-12-30.
- [91] FutureLearn, "The psychology of games," <https://www.futurelearn.com/info/courses/game-psychology/0/steps/428462>, n.d., accessed: 2025-12-31.
- [92] Red White Console, "Motivation and games: Attribution theory," <https://www.redwhiteconsole.net/motivation-games-attribution-theory/>, n.d., accessed: 2025-12-31.
- [93] ScienceDirect, "Expectancy-value theory," <https://www.sciencedirect.com/topics/psychology/expectancy-value-theory>, n.d., accessed: 2025-12-31.
- [94] A. Yu, "Improving the streak: Forming habits one lesson at a time," <https://blog.duolingo.com/improving-the-streak/>, n.d., accessed: 2025-12-31.
- [95] D. Jin, "Diplomarbeit," 2025, internal Reference. Accessed: 2025-12-31.

- [96] A. Marczewski, "Feedback loops, gamification and employee motivation," <https://www.gamified.uk/2013/03/25/feedback-loops-gamification-and-employee-motivation/>, 2013, accessed: 2025-12-31.
- [97] Motivacraft, "Feedback loops in gamification: The key to engaging user experiences," <https://www.motivacraft.com/feedback-loops-in-gamification-the-key-to-engaging-user-experiences/>, n.d., accessed: 2025-12-31.
- [98] Texas A&M AgriLife Extension (Aggie Horticulture), "Efficient use of water in the garden and landscape," <https://aggie-horticulture.tamu.edu/earthkind/drought/efficient-use-of-water-in-the-garden-and-landscape/>, n.d., accessed: 2025-12-31.
- [99] Iowa State University, "Soil aeration – introduction to soil science," <https://iastate.pressbooks.pub/isudp-2025-201/chapter/soil-aeration/>, 2026, accessed: 2026-02-24.
- [100] Cornell University Composting, "Oxygen diffusion," <https://compost.css.cornell.edu/oxygen/oxygen.diff.html>, 2026, accessed: 2026-02-24.
- [101] K. J. McCree, "Test of current definitions of photosynthetically active radiation against leaf photosynthesis data," *Agricultural Meteorology*, vol. 10, pp. 443–453, 1972.
- [102] U.S. Geological Survey (USGS), "Evapotranspiration and the water cycle," <https://www.usgs.gov/water-science-school/science/evapotranspiration-and-water-cycle>, n.d., accessed: 2026-02-24.
- [103] Encyclopaedia Britannica, "Turgor," <https://www.britannica.com/science/turgor>, 2026, accessed: 2026-02-24.